Relación entre características digitales y calidad perceptual de fotografías de teléfonos inteligentes

Estudio mediante técnicas de aprendizaje supervisado y no supervisado

Curso: Ciencia de Datos

Maestría en Inteligencia Artificial

Universidad de La Salle

Estudiante: Carlos Mario Garzón

Docente: Fabián Camilo Castro

Bogotá · 8 de noviembre de 2026

Resumen

La calidad perceptual de una fotografía depende de sus propiedades técnicas, del contenido representado y de la valoración subjetiva de los observadores, influenciada por sus experiencias, preferencias y criterios individuales. Aunque los descriptores digitales permiten cuantificar características tonales, cromáticas y espaciales de las imágenes, estas variables representan solo una parte de los factores que intervienen en la valoración humana. El objetivo de este trabajo fue evaluar la capacidad de un conjunto seleccionado de descriptores digitales para predecir la calidad perceptual de fotografías capturadas con teléfonos inteligentes y analizar cómo varía dicha capacidad según características digitales, el contenido de escena y las propiedades visuales de las imágenes. Se analizaron 11.125 fotografías del conjunto *Smartphone Photography Attribute and Quality* (SPAQ), cuya calidad perceptual se expresa mediante puntuaciones medias de opinión (*Mean Opinion Score*, MOS) obtenidas a partir de evaluaciones humanas. Se emplearon catorce descriptores interpretables y se compararon modelos de regresión lineales y no lineales, clasificadores multietiqueta de escenas y algoritmos de agrupamiento no supervisado. Los datos se dividieron en conjuntos de entrenamiento, validación y prueba, evitando que fotografías digitalmente idénticas aparecieran en subconjuntos diferentes. En el conjunto de validación (1.365 fotografías), *Random Forest* alcanzó un error absoluto medio de 8,9 puntos en la escala MOS de 0 a 100 y un coeficiente de determinación (R^2) de 0,684, correspondiente al 68,4 % de la variabilidad de las puntuaciones humanas. Esto representa una reducción del error del 50,3 % respecto de un predictor constante y del 28,6 % frente a la regresión Ridge, que obtuvo un error de 12,5 puntos y un R^2 de 0,453, correspondiente al 45,3 % de la variabilidad. El desempeño de Random Forest demuestra que los descriptores digitales seleccionados contienen información predictiva considerable sobre la calidad perceptual. Sin embargo, la dispersión entre las puntuaciones observadas y predichas, junto con las diferencias de error según el nivel de calidad y el contenido de escena, evidencia que esta relación es heterogénea. Los resultados son compatibles con la naturaleza multidimensional de la valoración fotográfica: mientras los descriptores estudiados cuantifican determinadas propiedades visuales, el juicio humano incorpora también aspectos estéticos, semánticos y subjetivos que estas variables no representan explícitamente. En consecuencia, el estudio establece tanto la utilidad predictiva de una representación digital relativamente reducida como sus límites para describir una valoración que integra criterios técnicos y estéticos.

1. Introducción

La calidad de una fotografía comprende tanto sus propiedades técnicas como la valoración perceptual que realizan los observadores. Características como la nitidez, el contraste, la reproducción tonal y la distribución del color pueden cuantificarse mediante descriptores calculados a partir de los píxeles. Sin embargo, la valoración humana también incorpora el contenido representado, la composición y criterios estéticos influidos por las experiencias y preferencias individuales. En consecuencia, las propiedades digitales de una imagen proporcionan información relevante sobre su calidad, pero no necesariamente representan todas las dimensiones que intervienen en su apreciación perceptual.

Esta distinción plantea un problema importante para la evaluación computacional de fotografías. Por una parte, los descriptores digitales permiten obtener mediciones objetivas y reproducibles de determinadas características visuales. Por otra, la calidad perceptual se expresa mediante valoraciones humanas que integran componentes técnicos y subjetivos. Establecer hasta qué punto ambas formas de caracterización se relacionan, y cuánta información sobre la valoración humana puede obtenerse de los descriptores digitales, constituye una cuestión relevante para el análisis de calidad de imagen.

El conjunto *Smartphone Photography Attribute and Quality* (SPAQ), presentado por Fang et al. (2020), proporciona una base experimental para estudiar estas relaciones en fotografías capturadas con teléfonos inteligentes. Incluye puntuaciones medias de opinión (*Mean Opinion Score*, MOS), obtenidas a partir de evaluaciones humanas, junto con anotaciones sobre atributos perceptuales y contenido de escena. A partir de las fotografías también pueden calcularse descriptores tonales, cromáticos y espaciales, lo que permite examinar estadísticamente su relación con MOS y desarrollar modelos de predicción basados en propiedades cuantificables de las imágenes.

Los estudios de asociación entre descriptores digitales y calidad perceptual permiten identificar características relacionadas con las valoraciones humanas. Sin embargo, la asociación individual de un descriptor con MOS no determina necesariamente su contribución cuando se combina con otras variables. La redundancia entre características, las posibles relaciones no lineales y las diferencias entre contenidos fotográficos pueden modificar tanto la capacidad predictiva de un modelo como la distribución de sus errores. Por ello, una evaluación basada exclusivamente en correlaciones individuales o métricas globales de regresión resulta insuficiente para caracterizar la relación entre las propiedades digitales de las fotografías y su calidad perceptual.

La brecha analítica que aborda este trabajo consiste en establecer qué capacidad predictiva ofrece una representación reducida e interpretable de las fotografías cuando sus características se consideran conjuntamente, cómo cambia esa capacidad entre diferentes niveles de calidad y contenidos de escena, y qué información adicional aporta el análisis de su organización multivariada. Se trata de una cuestión específica de la representación estudiada en SPAQ, sin presuponer que constituye una carencia inédita en la literatura internacional sobre evaluación de calidad de imagen.

A partir de este planteamiento, la pregunta de investigación es: ¿en qué medida los descriptores digitales seleccionados permiten predecir la calidad perceptual de fotografías capturadas con teléfonos inteligentes y cómo varía esa capacidad según el nivel de calidad, el contenido de escena y las propiedades visuales de las imágenes?

El objetivo de este trabajo fue evaluar la capacidad predictiva de catorce descriptores digitales interpretables mediante la comparación de modelos de regresión lineales y no lineales, el análisis de sus errores y la caracterización del contenido y la organización de las fotografías mediante clasificación multietiqueta y agrupamiento no supervisado. La contribución del estudio consiste en integrar estas perspectivas para cuantificar la utilidad predictiva de los descriptores, identificar las condiciones en las que las estimaciones presentan mayores discrepancias y examinar las relaciones entre las propiedades digitales, el contenido fotográfico y las valoraciones humanas. De esta manera, se busca establecer tanto el alcance como las limitaciones de una representación basada en características tonales, cromáticas y espaciales para describir una variable perceptual que incorpora dimensiones técnicas y subjetivas.

2. Materiales y métodos

2.1. Observaciones, variables y partición

Se analizaron 11.125 fotografías JPEG de SPAQ, acompañadas de MOS en escala de 0 a 100, cinco calificaciones humanas auxiliares, nueve pertenencias de escena no excluyentes y metadatos de adquisición. A partir de las imágenes se calcularon 68 descriptores a diferentes escalas. Para los modelos principales se emplearon catorce características de tono, color, textura, estructura espacial y variación regional; entre ellas, la heterogeneidad tonal relativa y la amplitud P95–P05 desarrolladas en el tratamiento de datos. Las calificaciones auxiliares y las etiquetas de escena no se utilizaron para predecir MOS. Los algoritmos de agrupamiento tampoco recibieron puntuaciones humanas: su relación con MOS se estudió exclusivamente después de formar los grupos.

Para evitar que dos registros con un archivo JPEG binariamente idéntico aparecieran en entrenamiento y evaluación, las particiones se definieron mediante huellas SHA-256. Se asignaron 8.364 observaciones a entrenamiento, 1.365 a validación y 1.396 a prueba, sin cruces de contenido binario entre conjuntos. La comparación y selección de modelos se basó en los datos de desarrollo. El conjunto de prueba había sido consultado en una evaluación final anterior; cualquier repetición de sus cálculos tuvo únicamente carácter de verificación reproducible de decisiones ya fijadas. Esta separación controla duplicados exactos, pero no equivale a estratificación por escenas o a independencia de contenidos visualmente parecidos.

2.2. Estrategia analítica

La variable principal de respuesta fue MOS. Se compararon la referencia lineal, Ridge, Random Forest Regressor (Breiman, 2001) y un perceptrón multicapa (MLP) implementado en PyTorch. Los parámetros se seleccionaron con validación cruzada agrupada por identidad del contenido y las transformaciones se ajustaron únicamente con los datos de entrenamiento correspondientes. Como análisis complementario se predijeron las nueve pertenencias simultáneas de escena mediante KNN y Random Forest, contrastados con una referencia logística. Para estudiar la estructura de los atributos independientemente de las valoraciones humanas se aplicaron K-Means, con evaluación de inercia y silueta (Rousseeuw, 1987), y DBSCAN (Ester et al., 1996); el análisis de componentes principales (PCA) facilitó su representación gráfica. Se examinaron posteriormente los perfiles digitales, MOS y escenas de los grupos.

La regresión se evaluó mediante error absoluto medio (MAE, en puntos MOS), raíz del error cuadrático medio (RMSE) y coeficiente de determinación (R^2). MAE expresa la discrepancia absoluta promedio; RMSE da mayor peso a las discrepancias grandes; R^2 cuantifica la reducción relativa del error cuadrático frente a una predicción constante basada en la media de las observaciones evaluadas. Para clasificación multietiqueta se calcularon precisión, sensibilidad, F1, área bajo la curva ROC y pérdida de Hamming, atendiendo a los tamaños de las escenas. La interpretación de los descriptores combinó correlaciones individuales, importancia por permutación del modelo y ablación de familias; estos procedimientos responden a preguntas distintas y no deben intercambiarse. Las figuras presentan tanto resúmenes como distribuciones, condicionamientos y ejemplos reales de fotografías.

3. Resultados

La comparación de los modelos permitió establecer cuánto mejora la estimación de MOS cuando los catorce descriptores se utilizan conjuntamente. En las mismas 1.365 fotografías de validación, Random Forest alcanzó MAE = 8,927 puntos y R^2 = 0,684, frente a MAE = 12,506 y R^2 = 0,453 de Ridge. Esto representa una reducción del error del 28,6 % respecto de Ridge y del 50,3 % frente al predictor constante (MAE = 17,965). El MLP obtuvo MAE = 9,750 y R^2 = 0,626. La Figura 1 compara las métricas y las diferencias por fotografía, de modo que la ganancia global pueda examinarse junto con su distribución. El mejor desempeño de Random Forest es compatible con relaciones no lineales entre atributos, sin demostrar mecanismos perceptuales causales (Breiman, 2001).

Regresión · validación	MAE (MOS)	RMSE (MOS)	R^2
Ridge	12,506	15,232	0,453
Random Forest	8,927	11,577	0,684
MLP PyTorch	9,750	12,601	0,626
Referencia lineal	12,506	15,231	0,453

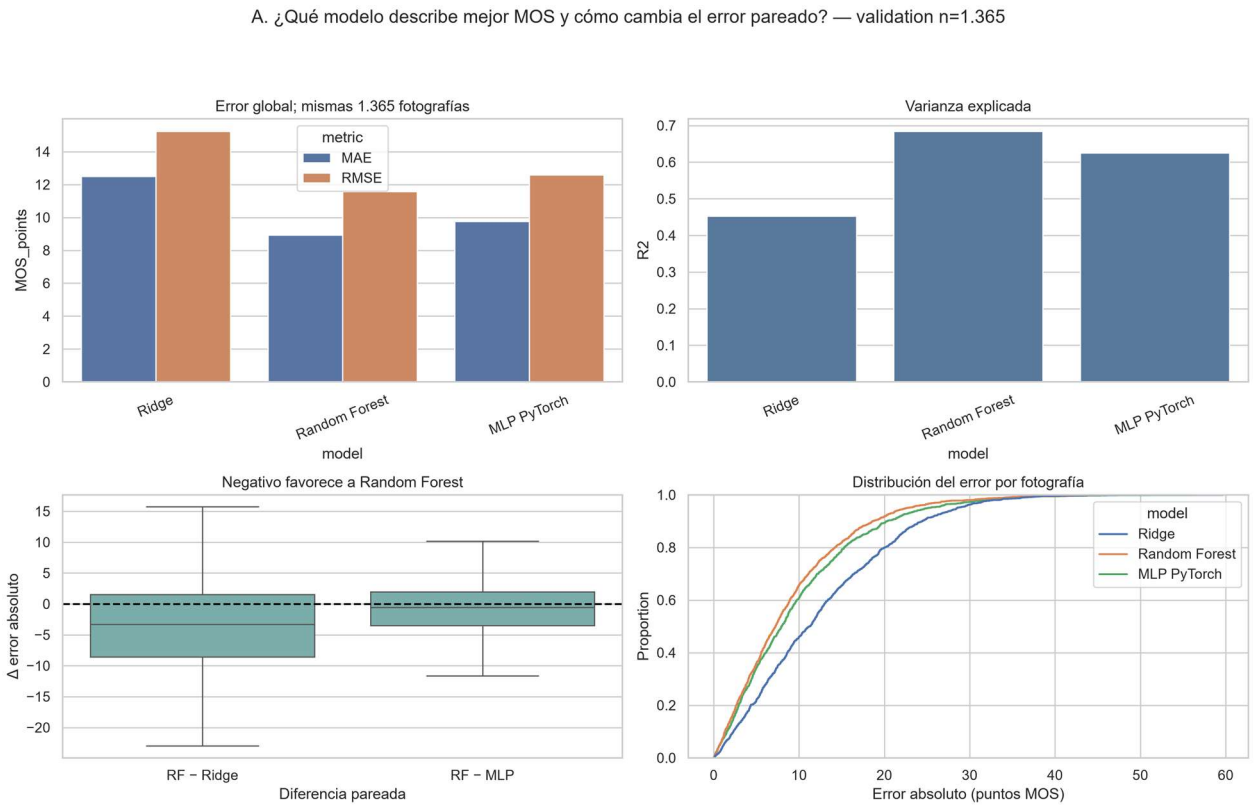


Figura 1. Comparación del error de regresión, varianza explicada y diferencias pareadas entre modelos. Validación: n = 1.365; errores en puntos MOS. Fuente: elaboración propia a partir de SPAQ.

La precisión del modelo varía a lo largo de la escala perceptual. En MOS 0–25 (179 fotografías), Random Forest obtuvo MAE = 11,163 puntos y residuo medio, definido como MOS observado menos predicho, de −10,672; en MOS 75–100 (187 fotografías), MAE = 11,304 y residuo medio = +11,215. En cambio, los intervalos 25–50 (524 fotografías) y 50–75 (475 fotografías) registraron MAE de 8,460 y 7,663. El signo de los residuos revela sobreestimación de puntuaciones bajas y subestimación de las altas. Las Figuras 2 y 3 distinguen la dispersión global de los errores y su variación condicionada por MOS y escena.

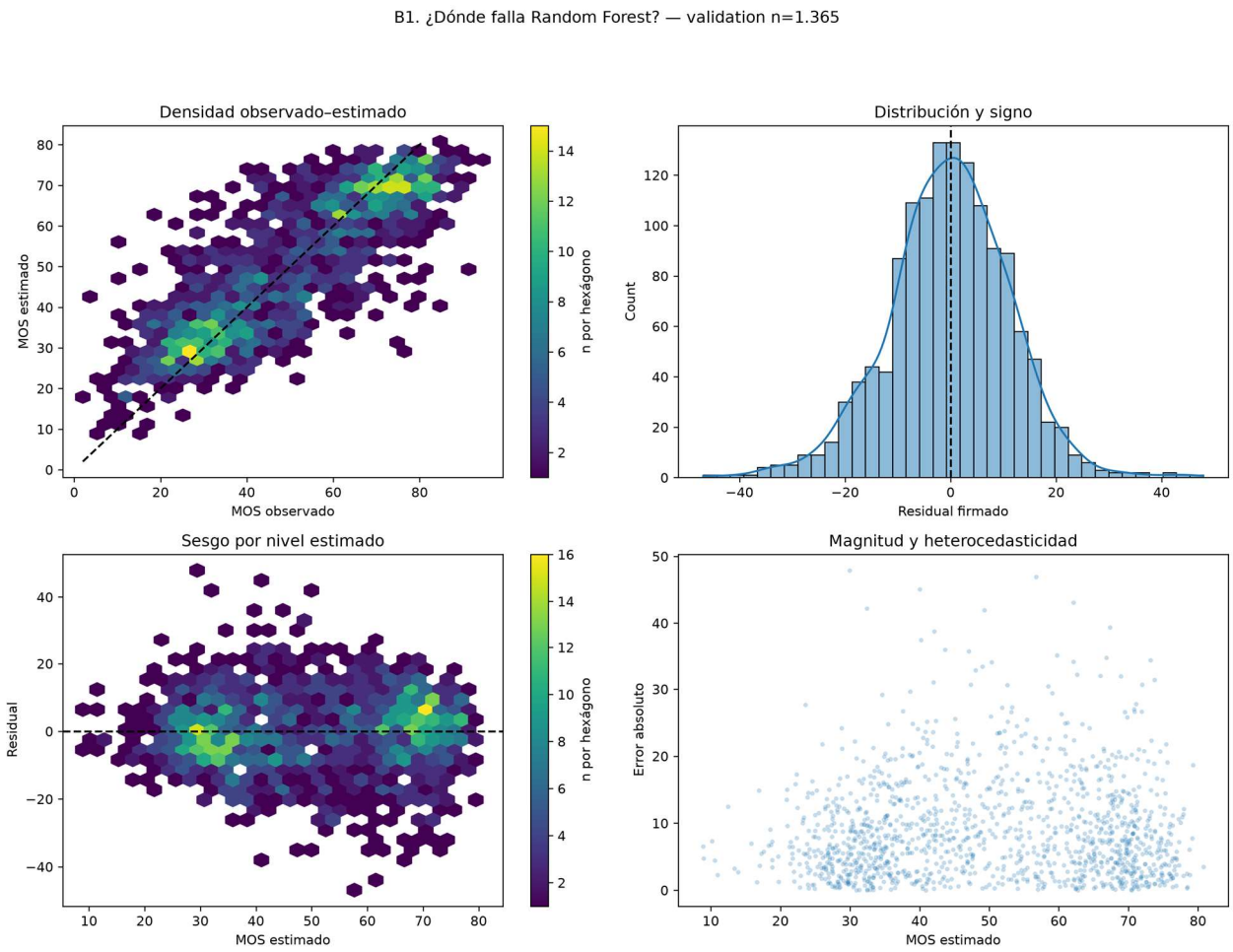


Figura 2. MOS observado y predicho por Random Forest y distribución global de residuos. Validación: n = 1.365; escala MOS 0–100. Fuente: elaboración propia a partir de SPAQ.

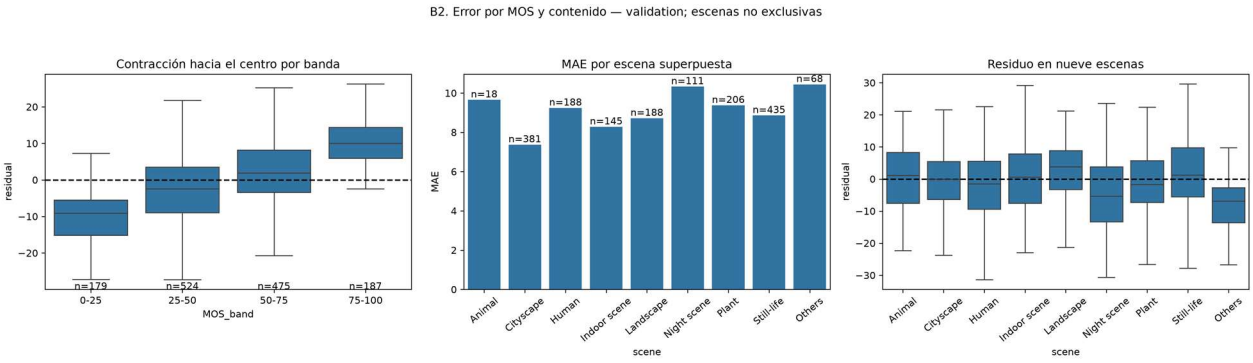


Figura 3. Residuos y errores absolutos de Random Forest condicionados por nivel MOS y pertenencia de escena. Validación; las escenas pueden superponerse. Fuente: elaboración propia a partir de SPAQ.

El contenido de escena también está asociado con diferencias de precisión (Figura 3). El MAE fue de 7,373 puntos en paisaje urbano, frente a 10,330 en escenas nocturnas y 10,441 en «otras». Dado que las categorías no son excluyentes y tienen distintos tamaños muestrales, estas diferencias describen heterogeneidad de desempeño y no efectos causales de la escena.

3.3. Reconocimiento de escenas: selectividad y recuperación

El reconocimiento de escenas se evaluó como clasificación multietiqueta, dado que una fotografía puede pertenecer a varias categorías. La Figura 4 compara precisión, sensibilidad, F1 y pérdida de Hamming entre los modelos. KNN obtuvo precisión micro = 0,562, sensibilidad micro = 0,377 y F1 micro = 0,451. Random Forest alcanzó mayor precisión (0,711), pero menor sensibilidad (0,228; F1 micro = 0,346). La regresión logística mostró el patrón opuesto, con sensibilidad = 0,724 y precisión = 0,297. Por ello, una menor pérdida de Hamming no determina por sí sola superioridad global.

La Figura 5 muestra la prevalencia de las nueve etiquetas, las decisiones y la sensibilidad al umbral. Para Animal, con 18 casos positivos en validación, KNN obtuvo F1 = 0,095 y AUC-ROC = 0,622; para Night scene, con 111 positivos, obtuvo F1 = 0,642 y AUC-ROC = 0,927. El menor soporte de algunas categorías y la elección del umbral obligan a interpretar las métricas por etiqueta junto con sus promedios agregados.

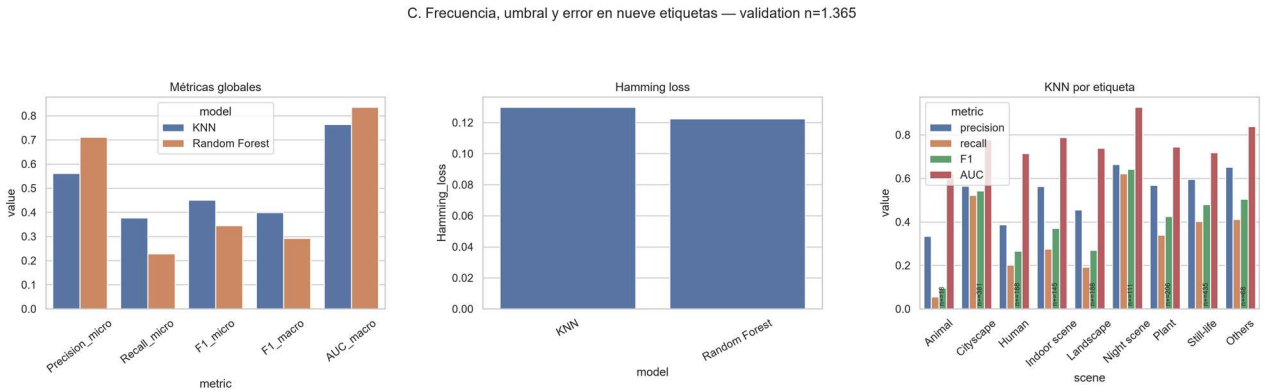


Figura 4. Comparación de clasificación multietiqueta: métricas agregadas y errores de decisión para las nueve escenas. Validación: n = 1.365. Fuente: elaboración propia a partir de SPAQ.

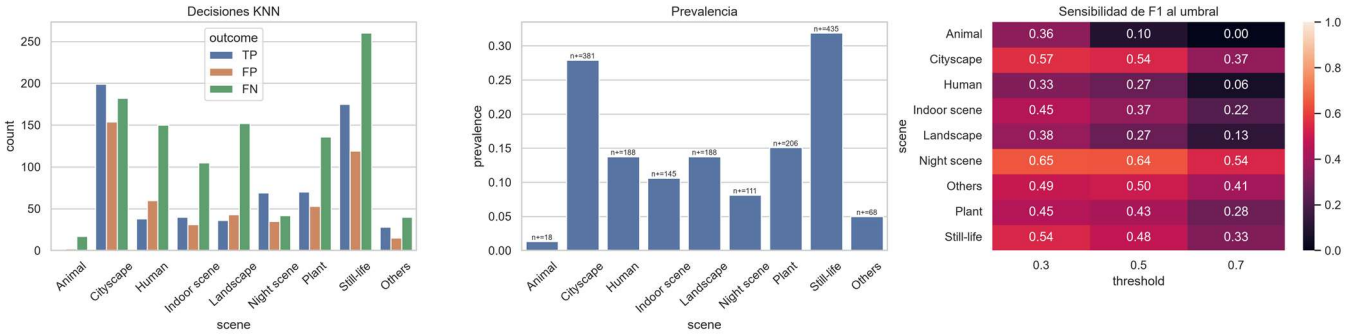


Figura 5. Prevalencia de escenas, desempeño por etiqueta y sensibilidad a umbrales de clasificación. Validación; interpretar según el soporte de cada escena. Fuente: elaboración propia a partir de SPAQ.

3.4. Organización multivariada: grupos y regiones de densidad

Para examinar la organización de las fotografías sin emplear MOS como entrada, se aplicó K-Means sobre los catorce atributos. La selección de K mediante inercia y silueta, junto con su proyección mediante componentes principales, se presentan en las Figuras 6 y 7. La mayor silueta entre los valores estudiados se obtuvo con K = 3 (0,232), valor que indica separación limitada (Rousseeuw, 1987). Los grupos incluyeron 1.313, 1.750 y 5.301 fotografías. Sus MOS medios, calculados solo después del agrupamiento, fueron 33,90, 69,49 y 48,16; la proyección PCA de dos componentes explicó el 52,6 % de la variabilidad estandarizada.

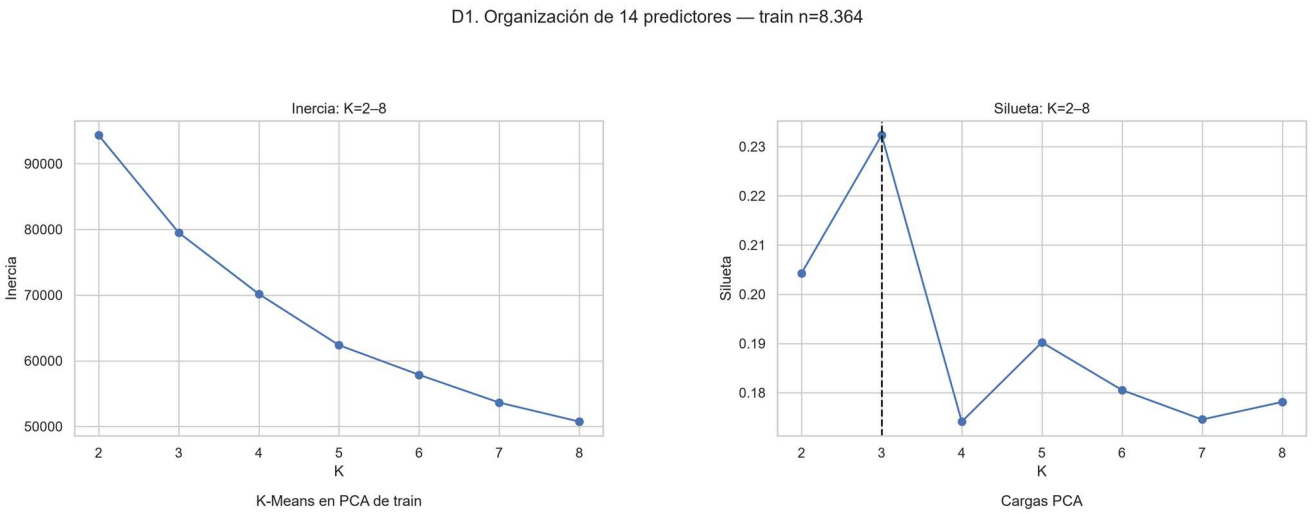


Figura 6. Selección del número de grupos mediante inercia y silueta de K-Means. Entrenamiento: n = 8.364; agrupamiento calculado sin MOS. Fuente: elaboración propia a partir de SPAQ.

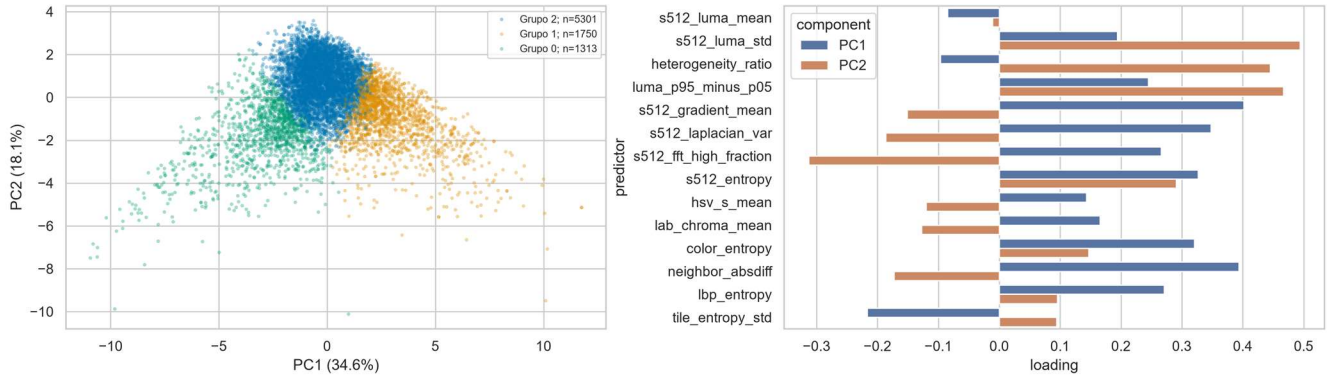


Figura 7. Proyección PCA de los grupos K-Means y contribuciones de atributos a los componentes. Entrenamiento; PC1–PC2 explican 52,6 % de la variación estandarizada. Fuente: elaboración propia a partir de SPAQ.

DBSCAN permitió contrastar la partición K-Means con una caracterización basada en densidad (Ester et al., 1996). Con eps = 1,2 y min_samples = 30 se identificó una región densa de 5.263 fotografías y 3.101 observaciones clasificadas como ruido, entre las 8.364 de entrenamiento (Figura 8). La restricción adicional sobre el tamaño final de los grupos fue una decisión operativa de interpretación, ajena a los requisitos matemáticos de DBSCAN. La Figura 9 presenta MOS y escenas como información posterior al agrupamiento. Los resultados describen regiones de diferente densidad, no categorías naturales universales.

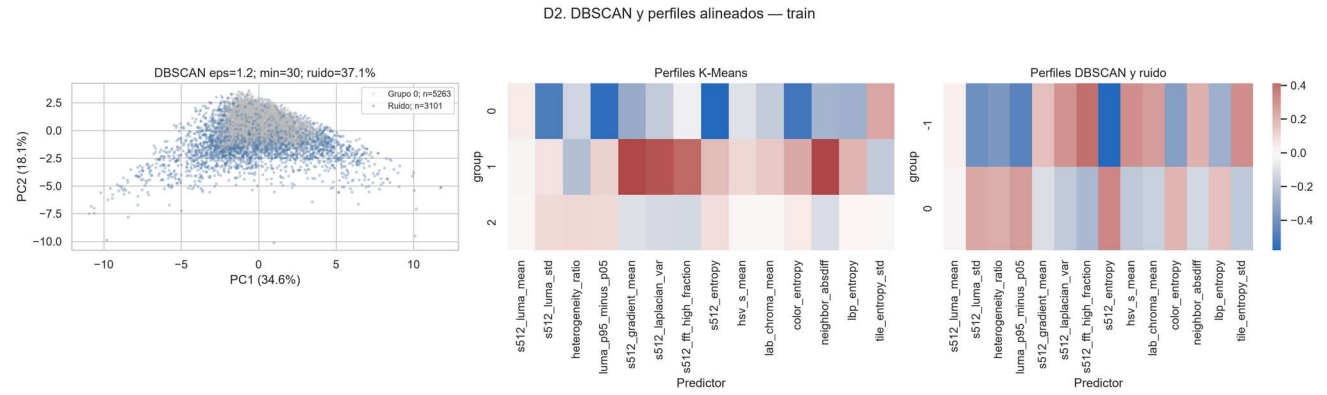


Figura 8. DBSCAN: región densa, observaciones clasificadas como ruido y perfiles de descriptores. Entrenamiento: n = 8.364. Fuente: elaboración propia a partir de SPAQ.

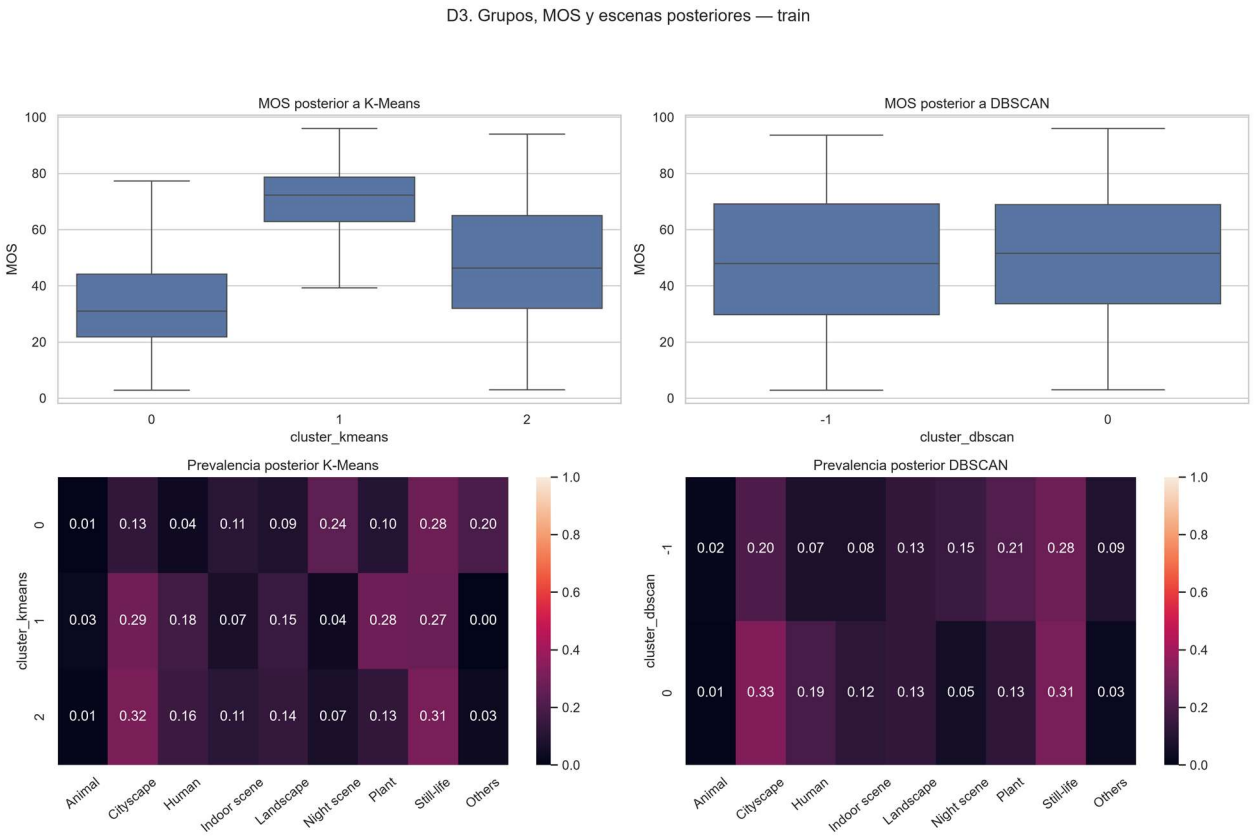


Figura 9. Distribuciones de MOS y prevalencia de escenas por agrupamiento. Ambas variables se añadieron después de calcular K-Means y DBSCAN. Fuente: elaboración propia a partir de SPAQ.

3.5. Importancia de características y consistencia de evidencias

La Figura 10 reúne cuatro contrastes diferentes de la información contenida en los atributos: importancia por permutación de Random Forest, asociación marginal, redundancia y ablación de familias en Ridge. La varianza del Laplaciano a escala 512 mostró tanto la mayor asociación de Spearman con MOS ($\rho \approx 0{,}736$) como la mayor importancia por permutación para Random Forest. La coincidencia apoya la relevancia de la estructura espacial, pero no constituye una medición óptica calibrada de nitidez. La correlación entre atributos condiciona la importancia obtenida y la ablación de Ridge no debe atribuirse a Random Forest.

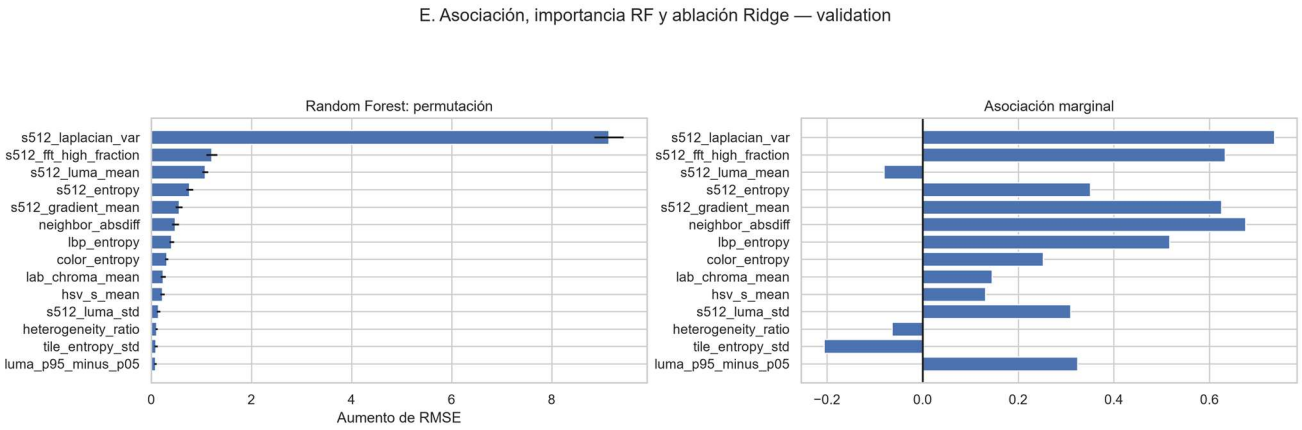


Figura 10. Importancia por permutación de Random Forest y correlaciones marginales entre descriptores y MOS. La importancia se mide como aumento de RMSE. Fuente: elaboración propia a partir de SPAQ.

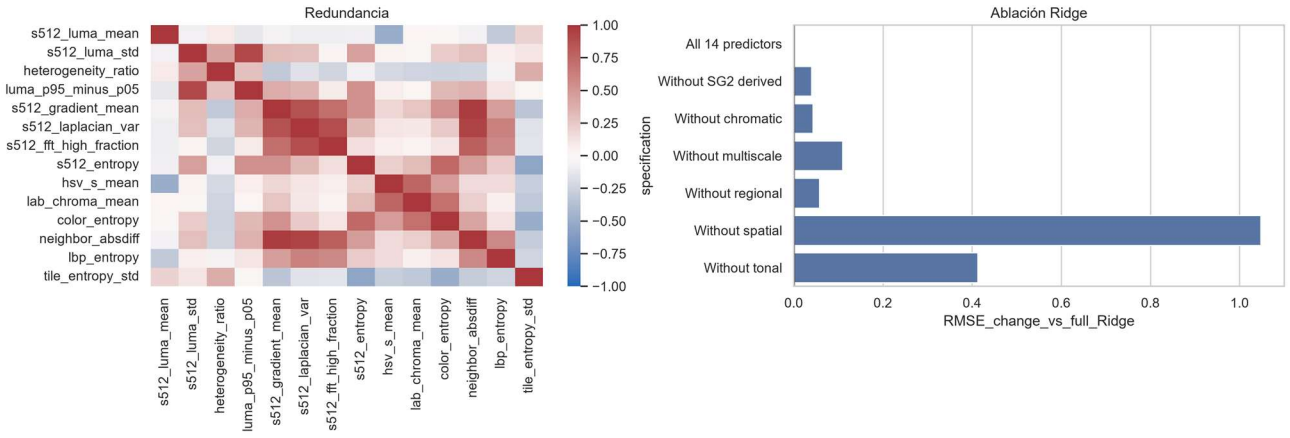


Figura 11. Redundancia entre los catorce predictores y ablación de familias en el modelo Ridge. La ablación no corresponde a Random Forest. Fuente: elaboración propia a partir de SPAQ.

3.6. Entrenamiento del MLP y evidencia a escala individual

El perceptrón multicapa (MLP) de PyTorch utilizó catorce entradas, dos capas ocultas de 32 y 16 unidades y una salida para estimar MOS. La Figura 12 muestra la pérdida durante el entrenamiento y la validación interna, incluida una ampliación de su etapa final. El mejor valor de control interno se alcanzó aproximadamente en la época 272. La mejora del ajuste no sustituye la evaluación en las 1.365 fotografías independientes de validación, donde el MLP obtuvo MAE = 9,750 puntos, mejor que Ridge, pero por detrás de Random Forest.

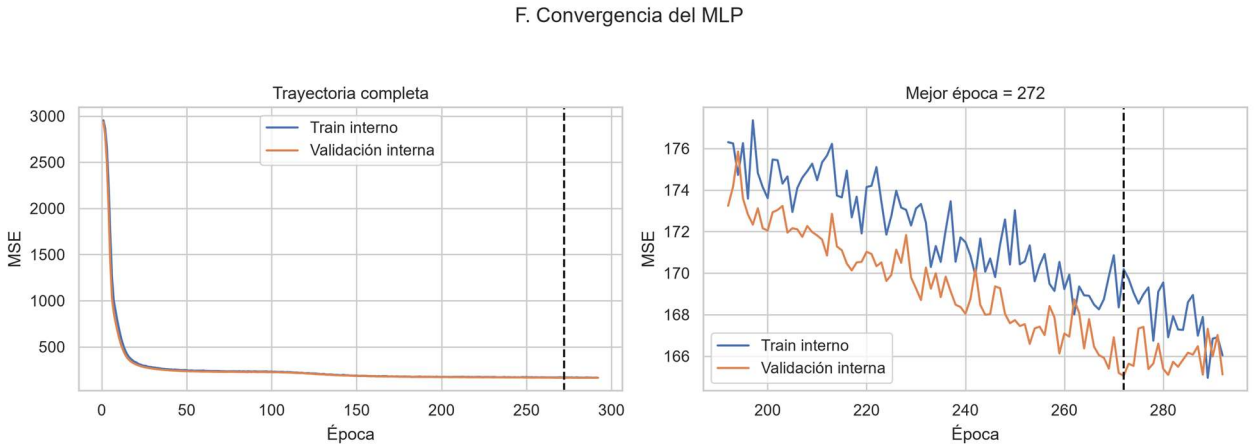


Figura 12. Evolución de la pérdida de entrenamiento y validación interna del MLP y ampliación de las épocas finales. Fuente: elaboración propia a partir de SPAQ.

Para relacionar las métricas globales con fotografías concretas se inspeccionaron cinco imágenes reales de validación seleccionadas mediante reglas reproducibles (Figura 13): puntuaciones MOS bajas y altas, errores de ambos signos y una predicción cercana a la observada. Los residuos extremos aproximados de −46,90 y +47,88 puntos son muy superiores al MAE global de 8,927. La Figura 14 muestra tres fotografías de entrenamiento próximas a los centroides de K-Means; son ejemplos de sus perfiles estadísticos y no categorías universales de calidad.



Figura 13. Fotografías originales de validación seleccionadas según MOS y magnitud y signo del residuo predictivo; se indican sus valores individuales. Fuente: elaboración propia a partir de SPAQ.



Figura 14. Fotografías originales de entrenamiento próximas a los centroides de los tres grupos K-Means. Fuente: elaboración propia a partir de SPAQ.

La comparación responde a la brecha principal: los catorce descriptores, cuyo interés inicial se apoyaba en asociaciones marginales y una regresión lineal, permitieron explicar el 68,4 % de la variabilidad de MOS en validación mediante Random Forest, con MAE de 8,927 puntos en una escala de 0 a 100. La reducción del MAE fue de 28,6 % respecto de Ridge y de 50,3 % frente al predictor constante. La diferencia entre los modelos lineales y el ensamble indica que las relaciones conjuntas no lineales contienen información predictiva que no queda aprovechada con igual precisión por una combinación lineal. Este es el avance cuantitativo del modelado respecto de la referencia inicial; no implica que los descriptores describan exhaustivamente el juicio humano.

La distribución de errores precisa el alcance de ese avance. En las bandas extremas de MOS, los MAE de 11,163 y 11,304 puntos superaron los 8,460 y 7,663 de las bandas centrales; además, las predicciones tendieron a elevar las calificaciones bajas y reducir las altas. Las diferencias de error entre escenas y los casos individuales con discrepancias próximas a 47 puntos demuestran que la capacidad predictiva no es homogénea. Esta evidencia es compatible con la naturaleza multidimensional de la valoración fotográfica: las catorce variables cuantifican determinadas propiedades visuales, mientras que el contenido semántico, la composición y los criterios individuales no están representados de forma explícita. La contribución de cada uno de estos factores a la dispersión residual no ha sido separada experimentalmente; tampoco se ha demostrado un umbral técnico general de aceptabilidad.

Los resultados complementarios especifican qué parte de la relación entre mediciones digitales y MOS puede examinarse desde distintos métodos. La coincidencia entre correlación individual e importancia por permutación destaca la estructura espacial, mientras que la separación limitada de K-Means y la región densa identificada por DBSCAN describen una organización multivariada sin imponer una taxonomía de calidad. La clasificación multietiqueta revela variaciones adicionales entre escenas. En conjunto, estas evidencias complementan —pero no sustituyen— la comparación de regresión y la caracterización de sus residuos.

4. Conclusiones

1. La representación seleccionada de catorce descriptores contiene información relevante para predecir la calidad perceptual de SPAQ. Random Forest obtuvo MAE = 8,927 puntos MOS y R^2 = 0,684 en validación, reduciendo el MAE un 28,6 % frente a Ridge y un 50,3 % frente al predictor constante. Frente a la referencia lineal, el avance consiste en cuantificar la utilidad adicional de relaciones no lineales entre atributos digitales.

2. La mejora predictiva no se distribuye uniformemente. En los intervalos MOS 0–25 y 75–100, los MAE fueron 11,163 y 11,304 puntos, superiores a los 8,460 y 7,663 de los intervalos centrales. La sobreestimación de puntuaciones bajas y la subestimación de las altas, junto con diferencias entre escenas, delimitan las condiciones en que una métrica global describe insuficientemente el comportamiento de las predicciones.

3. Los análisis multivariados aportaron una descripción complementaria de la representación visual. K-Means identificó tres perfiles con MOS medios posteriores de 33,90, 69,49 y 48,16, aunque con separación limitada (silueta = 0,232); DBSCAN distinguió una región densa de 5.263 fotografías y 3.101 observaciones dispersas. La asociación del Laplaciano con MOS y su importancia predictiva refuerzan la relevancia estadística de los atributos espaciales, sin establecer categorías naturales de calidad ni relaciones causales.

4. El conocimiento aportado por el estudio es doble: una representación digital reducida permite predecir una proporción considerable de la variación de una valoración humana compleja, pero conserva errores importantes y heterogéneos. Esta coexistencia es consistente con la diferencia entre las propiedades técnicas y estadísticas medidas y el conjunto más amplio de dimensiones perceptuales, semánticas y estéticas de MOS. La evaluación integrada de modelos, errores, escenas, agrupamientos y fotografías individuales proporciona evidencia cuantitativa para interpretar esa correspondencia y una base trazable para la exploración interactiva posterior.

Referencias

Breiman, L. (2001). Random forests. Machine Learning, 45, 5–32. https://doi.org/10.1023/A:1010933404324

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. Proceedings of the Second International Conference on Knowledge Discovery and Data Mining, 226–231.

Fang, Y., Zhu, H., Zeng, Y., Ma, K., & Wang, Z. (2020). Perceptual quality assessment of smartphone photography. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 3677–3686. https://doi.org/10.1109/CVPR42600.2020.00373

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. Journal of Computational and Applied Mathematics, 20, 53–65. https://doi.org/10.1016/0377-0427(87)90125-7

SPAQ (fotografías y anotaciones originales). Repositorio de los autores: https://github.com/h4nwei/SPAQ.

ANEXO

Print PDF del notebook usado

Estudio del dataset Smartphone Photography Attribute and Quality - Fase 3

Reconstrucción reproducible desde 11.125 JPEG y tres Excel originales. Conserva los 68 descriptores, 14 predictores, partición SHA, modelos y análisis aprobados.

In [1]

```
import hashlib, io, json, platform, random, time, warnings
from concurrent.futures import ThreadPoolExecutor
from datetime import datetime
from pathlib import Path
from zoneinfo import ZoneInfo
import cv2, joblib, numpy as np, pandas as pd, scipy, seaborn as sns, sklearn, torch
import matplotlib.pyplot as plt
from IPython.display import display
from PIL import Image, ImageCms, ImageOps
from joblib import parallel_backend
from scipy.stats import spearmanr
from skimage.feature import local_binary_pattern
from sklearn import base, cluster, decomposition, ensemble, impute, inspection
from sklearn import linear_model, metrics as skm, model_selection, neighbors, pipeline, preprocessing
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

ROOT, SEED, WORKERS = Path(r"D:\Data analisys"), 20261115, 16
SPAQ, PACKAGE = ROOT / "spaq/SPAQ", ROOT / "SPAQ_FASE3_COMPACTO"
ANNOTATIONS, IMAGES = SPAQ / "Annotations", SPAQ / "TestImage"
RESULTS, FIGURES = PACKAGE / "resultados_sg3", PACKAGE / "resultados_sg3/figuras"
MODELS = PACKAGE / "resultados_sg3/modelos"
for folder in (PACKAGE, RESULTS, FIGURES, MODELS): folder.mkdir(parents=True, exist_ok=True)
FILES = {name: ANNOTATIONS / filename for name, filename in {"scores": "MOS and Image attribute scores.xlsx",
    "scenes": "Scene category labels.xlsx", "exif": "EXIF_tags.xlsx"}.items()}
SCENES = "Animal|Cityscape|Human|Indoor scene|Landscape|Night scene|Plant|Still-life|Others".split("|")
PREDICTORS = """s512_luma_mean s512_luma_std heterogeneity_ratio luma_p95_minus_p05
s512_gradient_mean s512_laplacian_var s512_fft_high_fraction s512_entropy hsv_s_mean
lab_chroma_mean color_entropy neighbor_absdiff lbp_entropy tile_entropy_std""".split()
FAMILY_GROUPS = {"tonal": "s512_luma_mean s512_luma_std luma_p95_minus_p05",
    "regional": "heterogeneity_ratio tile_entropy_std",
    "spatial": "s512_gradient_mean s512_laplacian_var neighbor_absdiff lbp_entropy",
    "multiscale": "s512_fft_high_fraction s512_entropy", "chromatic": "hsv_s_mean lab_chroma_mean color_entropy"}
FAMILIES = {name: family for family, names in FAMILY_GROUPS.items() for name in names.split()}
BASE = "luma_mean luma_std gradient_mean laplacian_var fft_high_fraction tile_mean_std entropy".split()
EXTRA = """analysis_width analysis_height luma_median luma_min luma_max luma_range luma_skew gradient_p95
tile_gradient_cv tile_dark_fraction tile_bright_fraction tile_entropy_mean tile_entropy_std hsv_s_mean hsv_s_std
hsv_v_mean rgb_r_mean rgb_g_mean rgb_b_mean rgb_r_minus_g rgb_b_minus_g lab_l_mean lab_a_mean lab_b_mean
lab_chroma_mean lab_chroma_std tile_chroma_std color_entropy tile_color_entropy_std lbp_entropy neighbor_absdiff
fft_centroid""".split()
DESCRIPTORS = ([f"s{s}_{name}" for s in (512, 256, 128) for name in BASE] + EXTRA
    + [f"luma_p{p:02}" for p in (1, 5, 25, 75, 95, 99)] + [f"near_black_{p:02}" for p in (1, 5, 10)]
    + [name for p in (245, 250, 254) for name in (f"near_white_{p}", f"channel_clip_{p}")])
for seed in (random.seed, np.random.seed, torch.manual_seed): seed(SEED)
warnings.filterwarnings("ignore", category=UserWarning); cv2.setNumThreads(1); sns.set_theme(style="whitegrid")
assert len(DESCRIPTORS) == 68 and len(PREDICTORS) == 14

def save(fig, name, title, top=.94):
    fig.suptitle(title, y=.99); fig.tight_layout(rect=(0, 0, 1, top))
    for suffix, kwargs in (("png", {"dpi": 240}), ("svg", {})):
```

```
        fig.savefig(FIGURES / f"{name}.{suffix}", bbox_inches="tight", **kwargs)
    plt.show()

def annotations(path, keep=None):
    frame = pd.read_excel(path); frame["image_id"] = frame["Image name"].map(lambda x: Path(str(x)).stem.zfill(5))
    return frame if keep is None else frame.loc[frame.image_id.isin(keep)].copy()

versions = {"date": datetime.now(ZoneInfo("America/Bogota")).isoformat(), "python": platform.python_version(),
    **{k: v.__version__ for k, v in {"numpy": np, "pandas": pd, "scipy": scipy, "opencv": cv2,
        "scikit_learn": sklearn, "torch": torch}.items()}}
(RESULTS / "versiones.json").write_text(json.dumps(versions, indent=2), encoding="utf-8")
```

```
208
```

Descriptores de imagen

In [2]

```
def entropy(values, bins=256):
    return float(scipy.stats.entropy(np.histogram(values, bins=bins, range=(0, 1))[0], base=2))

def color_entropy(rgb):
    q = np.minimum((rgb * 8).astype(np.int32), 7)
    return float(scipy.stats.entropy(np.bincount((q[:, :, 0] * 64 + q[:, :, 1] * 8 + q[:, :, 2]).ravel(),
        minlength=512), base=2))

def tiles(values):
    return [tile for row in np.array_split(values, 4, axis=0) for tile in np.array_split(row, 4, axis=1)]

def spectrum(luma):
    height, width = luma.shape
    power = np.abs(np.fft.rfft2((luma - luma.mean()) * np.outer(np.hanning(height), np.hanning(width)),
        norm="ortho")) ** 2
    weights = np.full(power.shape[1], 2.); weights[0] = 1
    if width % 2 == 0: weights[-1] = 1
    power *= weights[None, :]; power[0, 0] = 0
    radius = np.hypot(np.fft.fftfreq(height)[:, None], np.fft.rfftfreq(width)[None, :]); total = power.sum()
    if total <= 1e-20: return None, None
    return float(power[radius >= .25].sum() / total), float((power * radius).sum() / total)

def extract_features(image):
    result = {}
    for scale in (512, 256, 128):
        width, height = image.size; factor = min(1, scale / max(width, height))
        size = max(1, round(width * factor)), max(1, round(height * factor))
        rgb = np.asarray(image.resize(size, Image.Resampling.LANCZOS), dtype=np.float32) / 255
        luma = rgb @ np.array([.2126, .7152, .0722], dtype=np.float32)
        gx, gy = (cv2.Sobel(luma, cv2.CV_32F, dx, dy, ksize=3) / 8 for dx, dy in ((1, 0), (0, 1)))
        gradient, luma_tiles = np.hypot(gx, gy), tiles(luma)
        tile_means, (high, centroid) = np.array([x.mean() for x in luma_tiles]), spectrum(luma)
        prefix = f"s{scale}_"
        result.update({prefix + "luma_mean": float(luma.mean()), prefix + "luma_std": float(luma.std()),
            prefix + "gradient_mean": float(gradient.mean()), prefix + "laplacian_var": float(
                cv2.Laplacian(luma, cv2.CV_32F, ksize=1).var()), prefix + "fft_high_fraction": high,
            prefix + "tile_mean_std": float(tile_means.std()), prefix + "entropy": entropy(luma)})
        if scale != 512: continue
        std = float(luma.std()); centered = (luma - luma.mean()) ** 3
```

```
        result.update({"analysis_width": luma.shape[1], "analysis_height": luma.shape[0],
            "luma_median": float(np.median(luma)), "luma_min": float(luma.min()), "luma_max": float(luma.max()),
            "luma_range": float(np.ptp(luma)), "luma_skew": float(centered.mean() / std**3) if std > 1e-8 else None})
        for p, value in zip((1, 5, 25, 75, 95, 99), np.percentile(luma, (1, 5, 25, 75, 95, 99))):
            result[f"luma_p{p:02}"] = float(value)
        for p in (1, 5, 10): result[f"near_black_{p:02}"] = float(np.mean(luma <= p / 255))
        for p in (245, 250, 254):
            result[f"near_white_{p}"] = float(np.mean(luma >= p / 255))
            result[f"channel_clip_{p}"] = float(np.mean(rgb.max(axis=2) >= p / 255))
        gradient_means = np.array([x.mean() for x in tiles(gradient)])
        tile_entropies = np.array([entropy(x) for x in luma_tiles])
        hsv, lab = cv2.cvtColor(rgb, cv2.COLOR_RGB2HSV), cv2.cvtColor(rgb, cv2.COLOR_RGB2Lab)
        chroma, rgb_mean = np.hypot(lab[:, :, 1], lab[:, :, 2]), rgb.mean(axis=(0, 1))
        differences = (np.abs(np.diff(luma, axis=0)).mean() + np.abs(np.diff(luma, axis=1)).mean()) / 2
        result.update({"gradient_p95": float(np.percentile(gradient, 95)),
            "tile_gradient_cv": float(gradient_means.std() / gradient_means.mean()) if gradient_means.mean() > 1e-8 else None,
            "tile_dark_fraction": float(np.mean(tile_means < .1)), "tile_bright_fraction": float(np.mean(tile_means > .9)),
            "tile_entropy_mean": float(tile_entropies.mean()), "tile_entropy_std": float(tile_entropies.std()),
            "fft_centroid": centroid, "hsv_s_mean": float(hsv[:, :, 1].mean()), "hsv_s_std": float(hsv[:, :, 1].std()),
            "hsv_v_mean": float(hsv[:, :, 2].mean()), "rgb_r_mean": float(rgb_mean[0]), "rgb_g_mean": float(rgb_mean[1]),
            "rgb_b_mean": float(rgb_mean[2]), "rgb_r_minus_g": float(rgb_mean[0] - rgb_mean[1]),
            "rgb_b_minus_g": float(rgb_mean[2] - rgb_mean[1]), "lab_l_mean": float(lab[:, :, 0].mean()),
            "lab_a_mean": float(lab[:, :, 1].mean()), "lab_b_mean": float(lab[:, :, 2].mean()),
            "lab_chroma_mean": float(chroma.mean()), "lab_chroma_std": float(chroma.std()),
            "tile_chroma_std": float(np.std([x.mean() for x in tiles(chroma)])), "color_entropy": color_entropy(rgb),
            "tile_color_entropy_std": float(np.std([color_entropy(x) for x in tiles(rgb)])),
            "neighbor_absdiff": float(differences)})
        lbp = local_binary_pattern(np.round(luma * 255).astype(np.uint8), 8, 1, "uniform")[1:-1, 1:-1]
        counts = np.bincount(lbp.astype(int).ravel(), minlength=10); probabilities = counts[counts > 0] / counts.sum()
        result["lbp_entropy"] = float(-(probabilities * np.log2(probabilities)).sum())
    assert set(result) == set(DESCRIPTORS)
    return result

SRGB = ImageCms.createProfile("sRGB")
def process_image(image_id):
    content = (IMAGES / f"{image_id}.jpg").read_bytes()
    with Image.open(io.BytesIO(content)) as raw:
        raw.load(); image = ImageOps.exif_transpose(raw); profile = raw.info.get("icc_profile")
        try:
            if profile: image = ImageCms.profileToProfile(image, ImageCms.ImageCmsProfile(io.BytesIO(profile)),
                SRGB, outputMode="RGB")
        except Exception: pass
        values = extract_features(image.convert("RGB"))
    return {"image_id": image_id, "sha256": hashlib.sha256(content).hexdigest(), **values}
```

Datos, SHA y partición

In [3]

```
score_ids = annotations(FILES["scores"])[["Image name", "image_id"]]
assert len(score_ids) == len(pd.read_excel(FILES["exif"])) == 11125
assert list(pd.read_excel(FILES["scenes"], nrows=0)) == ["Image name", *SCENES]
image_ids = sorted(score_ids.image_id); pilot = pd.DataFrame(map(process_image, image_ids[:5]))
assert pilot.shape == (5, 70) and pilot[DESCRIPTORS].notna().all().all(); display(pilot.iloc[:, :4])
start = time.time()
with ThreadPoolExecutor(max_workers=WORKERS) as executor: images = pd.DataFrame(executor.map(process_image, image_ids))
extraction_minutes = (time.time() - start) / 60
```

```
assert len(images) == 11125 and images.sha256.nunique() == 10803
def split_sha(sha):
    value = int(hashlib.sha256(f"{SEED}|{sha}".encode()).hexdigest()[:16], 16) / 2**64
    return "train" if value < .75 else "validation" if value < .875 else "test"
metadata = images.assign(heterogeneity_ratio=images.s512_tile_mean_std / images.s512_luma_std,
    luma_p95_minus_p05=images.luma_p95 - images.luma_p05, split=images.sha256.map(split_sha))
metadata.loc[metadata.s512_luma_std <= 1e-8, "heterogeneity_ratio"] = np.nan
metadata["sha_weight"] = 1 / metadata.sha256.map(metadata.groupby("sha256").size())
development_ids = set(metadata.loc[metadata.split != "test", "image_id"])
development = annotations(FILES["scores"], development_ids).merge(annotations(FILES["scenes"], development_ids),
    on=["image_id", "Image name"], validate="one_to_one").merge(metadata.query("split != 'test'"), on="image_id")
TARGETS = [f"target_{scene}" for scene in SCENES]
development[TARGETS] = (development[SCENES] > 0).astype("int8").set_axis(TARGETS, axis=1)
train, validation = [development.query("split == @name").copy() for name in ("train", "validation")]
test_sealed = metadata.query("split == 'test'").copy(); split_counts = metadata.split.value_counts()
assert split_counts.to_dict() == {"train": 8364, "test": 1396, "validation": 1365}
assert metadata.groupby("sha256").split.nunique().max() == 1 and "MOS" not in test_sealed
X_train, y_train, Y_train = train[PREDICTORS], train.MOS, train[TARGETS].to_numpy()
X_validation, y_validation, Y_validation = validation[PREDICTORS], validation.MOS, validation[TARGETS].to_numpy()
groups_train, group_cv = train.sha256, model_selection.GroupKFold(3)
```

	image_id	sha256	s512_luma_mean	s512_luma_std
0	00001	0a1dcb4b6759130f41a089eaafa53176463f02141f685d...	0.460585	0.251393
1	00002	a01f7dc1429764a0c2c44f2024473c1dee74ab83f8241a...	0.318680	0.217884
2	00003	04951d9704ec68917896f79ca7fc8667833fa976f30230...	0.426044	0.292064
3	00004	e973f536a8955cc62d9b424e926ff92b068fc4aebec831...	0.421824	0.244125
4	00005	e2406ffe8d91cba5495229b28a65adb34e7f895d044742...	0.359360	0.250307

Regresión y clasificación multietiqueta

In [4]

```
def reg_metrics(y, prediction):
    return {"MAE": skm.mean_absolute_error(y, prediction), "RMSE": skm.mean_squared_error(y, prediction) ** .5,
        "R2": skm.r2_score(y, prediction)}
def reg_table(frame, prediction):
    return frame[["image_id", "sha256", "split", "MOS", *SCENES]].copy().assign(MOS_predicted=prediction,
        residual=lambda x: x.MOS - x.MOS_predicted, absolute_error=lambda x: x.residual.abs())
def scores(model, values): return np.column_stack([x[:, 1] for x in model.predict_proba(values)])
def class_metrics(y, prediction, score):
    precision, recall, f1, _ = skm.precision_recall_fscore_support(y, prediction, average="micro", zero_division=0)
    return {"F1_micro": f1, "F1_macro": skm.f1_score(y, prediction, average="macro", zero_division=0),
        "Precision_micro": precision, "Recall_micro": recall, "Hamming_loss": skm.hamming_loss(y, prediction),
        "Subset_accuracy": skm.accuracy_score(y, prediction), "AUC_macro": skm.roc_auc_score(y, score, average="macro")}
def class_table(frame, y, prediction, score):
    blocks = [frame[["image_id", "sha256", "split"]].reset_index(drop=True)]
    blocks += [pd.DataFrame(x, columns=SCENES).add_prefix(prefix) for x, prefix in
        ((y, "observed_"), (prediction, "predicted_"), (score, "score_"))]
    return pd.concat(blocks, axis=1)
def search(estimator, parameters, scoring, y):
    grid = model_selection.GridSearchCV(estimator, parameters, scoring=scoring, cv=group_cv, n_jobs=-1)
    with parallel_backend("threading", n_jobs=WORKERS): grid.fit(X_train, y, groups=groups_train)
```

```
    return grid

ridge = search(pipeline.make_pipeline(impute.SimpleImputer(strategy="median"), preprocessing.RobustScaler(),
    linear_model.Ridge()), {"ridge__alpha": [.1, 1., 10., 100.]}, "neg_root_mean_squared_error", y_train)
forest = search(pipeline.make_pipeline(impute.SimpleImputer(strategy="median"),
    ensemble.RandomForestRegressor(random_state=SEED, n_jobs=1)),
    {"randomforestregressor__n_estimators": [250], "randomforestregressor__max_depth": [None, 12],
     "randomforestregressor__min_samples_leaf": [1, 4], "randomforestregressor__max_features": ["sqrt", .7]},
    "neg_root_mean_squared_error", y_train)
knn = search(pipeline.make_pipeline(impute.SimpleImputer(strategy="median"), preprocessing.RobustScaler(),
    neighbors.KNeighborsClassifier()), {"kneighborsclassifier__n_neighbors": [5, 11, 21],
    "kneighborsclassifier__weights": ["uniform", "distance"], "kneighborsclassifier__p": [1, 2]}, "f1_micro", Y_train)
classifier = search(pipeline.make_pipeline(impute.SimpleImputer(strategy="median"), ensemble.RandomForestClassifier(
    random_state=SEED, n_jobs=1, class_weight="balanced_subsample")),
    {"randomforestclassifier__n_estimators": [250], "randomforestclassifier__max_depth": [None, 16],
     "randomforestclassifier__min_samples_leaf": [1, 4], "randomforestclassifier__max_features": ["sqrt", .7]},
    "f1_micro", Y_train)
regression_models = {"Ridge": ridge.best_estimator_, "Random Forest": forest.best_estimator_}
classification_models = {"KNN": knn.best_estimator_, "Random Forest": classifier.best_estimator_}
regression_predictions = {name: model.predict(X_validation) for name, model in regression_models.items()}
classification_predictions = {name: model.predict(X_validation) for name, model in classification_models.items()}
classification_scores = {name: scores(model, X_validation) for name, model in classification_models.items()}
regression_summary = pd.DataFrame([{"model": name, **reg_metrics(y_validation, value)}
    for name, value in regression_predictions.items()])
classification_summary = pd.DataFrame([{"model": name, **class_metrics(Y_validation,
    classification_predictions[name], classification_scores[name])} for name in classification_models])
frozen = {"regressor": "Random Forest", "regressor_parameters": forest.best_params_, "classifier": "KNN",
    "classifier_parameters": knn.best_params_}; display(regression_summary, classification_summary)
```

	model	MAE	RMSE	R2
0	Ridge	12.506245	15.231568	0.452882
1	Random Forest	8.926698	11.576659	0.683948

	model	F1_micro	F1_macro	Precision_micro	Recall_micro	Hamming_loss	Subset_accuracy	AUC_macro
0	KNN	0.451169	0.399472	0.561644	0.377011	0.129915	0.291575	0.763520
1	Random Forest	0.345518	0.292633	0.711470	0.228161	0.122426	0.206593	0.835444

Agrupamiento y PCA en entrenamiento

In [5]

```
cluster_preprocessor = pipeline.make_pipeline(impute.SimpleImputer(strategy="median"), preprocessing.StandardScaler())
Z_train = cluster_preprocessor.fit_transform(X_train)
k_selection = pd.DataFrame([{"K": k, "model": model, "inertia": model.inertia_,
    "silhouette": skm.silhouette_score(Z_train, labels)} for k in range(2, 9)
    for model in [cluster.KMeans(k, n_init=20, random_state=SEED).fit(Z_train)] for labels in [model.labels_]])
selected_k = int(k_selection.sort_values(["silhouette", "K"], ascending=[False, True]).iloc[0].K)
kmeans = cluster.KMeans(selected_k, n_init=30, random_state=SEED).fit(Z_train); kmeans_labels = kmeans.labels_
pca_two = decomposition.PCA(2, random_state=SEED).fit(Z_train); pca_coordinates = pca_two.transform(Z_train)
pca_loadings = pd.DataFrame(pca_two.components_.T, index=PREDICTORS, columns=["PC1", "PC2"])
pca_loadings["max_absolute_loading"] = pca_loadings.abs().max(axis=1)
Z_density = decomposition.PCA(n_components=.95, random_state=SEED).fit_transform(Z_train)
```

```
dbscan_rows = []
for eps in (1.2, 1.5, 1.8, 2.1):
    for minimum in (15, 30):
        candidate = cluster.DBSCAN(eps=eps, min_samples=minimum, n_jobs=-1).fit(Z_density); labels = candidate.labels_
        values, counts = np.unique(labels, return_counts=True); non_noise = counts[values != -1]; n = len(values) - (-1 in values)
        silhouette = skm.silhouette_score(Z_density[labels != -1], labels[labels != -1]) if n >= 2 else np.nan
        dbscan_rows.append({"eps": eps, "min_samples": minimum, "clusters": n, "noise_fraction": np.mean(labels == -1),
            "silhouette_non_noise": silhouette, "smallest_cluster": non_noise.min(),
            "all_cluster_sizes_valid": non_noise.min() >= minimum})
dbscan_selection = pd.DataFrame(dbscan_rows)
valid = dbscan_selection.query("clusters >= 2 and clusters <= 12 and noise_fraction <= .4 and all_cluster_sizes_valid")
if len(valid): choice = valid.sort_values("silhouette_non_noise", ascending=False).iloc[0]
else: choice = dbscan_selection.query("clusters >= 1 and noise_fraction <= .4 and all_cluster_sizes_valid").sort_values(
    ["eps", "min_samples"]).iloc[0]
dbscan = cluster.DBSCAN(eps=float(choice.eps), min_samples=int(choice.min_samples), n_jobs=-1).fit(Z_density)
dbscan_labels = dbscan.labels_; final_counts = np.unique(dbscan_labels, return_counts=True)
assert selected_k == 3 and all(count >= dbscan.min_samples for label, count in zip(*final_counts) if label != -1)
def profiles(labels, name):
    original = train[PREDICTORS].assign(group=labels).groupby("group").mean()
    standardized = pd.DataFrame(Z_train, columns=PREDICTORS).assign(group=labels).groupby("group").mean().add_prefix("z_")
    counts = pd.Series(labels).value_counts().sort_index()
    if name == "cluster_kmeans":
        original["n"], original["MOS_mean_posthoc"] = counts, train.assign(group=labels).groupby("group").MOS.mean()
    else: original.insert(0, "n", counts)
    result = original.join(standardized)
    return result
kmeans_profiles, dbscan_profiles = profiles(kmeans_labels, "cluster_kmeans"), profiles(dbscan_labels, "cluster_dbscan")
```

Perceptrón multicapa PyTorch

In [6]

```
fit_index, stop_index = next(model_selection.GroupShuffleSplit(1, test_size=.18, random_state=SEED).split(
    X_train, y_train, groups_train))
mlp_preprocessor = pipeline.make_pipeline(impute.SimpleImputer(strategy="median"), preprocessing.RobustScaler())
X_fit = mlp_preprocessor.fit_transform(X_train.iloc[fit_index]).astype("float32")
X_stop = mlp_preprocessor.transform(X_train.iloc[stop_index]).astype("float32")
y_fit, y_stop = y_train.iloc[fit_index].to_numpy("float32"), y_train.iloc[stop_index].to_numpy("float32")
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
mlp = nn.Sequential(nn.Linear(14, 32), nn.ReLU(), nn.Dropout(.1), nn.Linear(32, 16), nn.ReLU(), nn.Linear(16, 1)).to(device)
optimizer, loss_function = torch.optim.Adam(mlp.parameters(), lr=.001, weight_decay=.0001), nn.MSELoss()
loader = DataLoader(TensorDataset(torch.tensor(X_fit), torch.tensor(y_fit)), batch_size=256, shuffle=True,
    generator=torch.Generator().manual_seed(SEED))
X_stop_tensor, y_stop_tensor = torch.tensor(X_stop, device=device), torch.tensor(y_stop, device=device)
history, best_loss, best_state, best_epoch, wait = [], np.inf, None, 0, 0
for epoch in range(1, 301):
    mlp.train(); accumulated = 0
    for X_batch, y_batch in loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device); optimizer.zero_grad()
        loss = loss_function(mlp(X_batch).squeeze(1), y_batch); loss.backward(); optimizer.step()
        accumulated += loss.item() * len(X_batch)
    mlp.eval()
    with torch.no_grad(): stop_loss = loss_function(mlp(X_stop_tensor).squeeze(1), y_stop_tensor).item()
    history.append({"epoch": epoch, "train_mse": accumulated / len(X_fit), "stop_mse": stop_loss})
    if stop_loss < best_loss - .0001:
        best_loss, best_epoch, wait = stop_loss, epoch, 0
        best_state = {name: value.detach().cpu().clone() for name, value in mlp.state_dict().items()}
```

```
    else: wait += 1
    if wait >= 20: break
mlp.load_state_dict(best_state); history = pd.DataFrame(history); mlp.eval()
with torch.no_grad():
    mlp_prediction = mlp(torch.tensor(mlp_preprocessor.transform(X_validation).astype("float32"),
        device=device)).squeeze(1).cpu().numpy()
regression_predictions["MLP PyTorch"] = mlp_prediction
regression_summary.loc[len(regression_summary)] = {"model": "MLP PyTorch", **reg_metrics(y_validation, mlp_prediction)}
```

Tablas analíticas e interpretabilidad

In [7]

```
rf_prediction = regression_predictions["Random Forest"]; regression_validation = reg_table(validation, rf_prediction)
regression_validation["MOS_band"] = pd.cut(regression_validation.MOS, [-np.inf, 25, 50, 75, np.inf],
    labels=["0-25", "25-50", "50-75", "75-100"])
slugs = {"Ridge": "Ridge", "Random Forest": "Random_Forest", "MLP PyTorch": "MLP_PyTorch"}
regression_finalists = validation[["image_id", "sha256", "split", "MOS", *SCENES]].copy()
errors = {}
for name, prediction in regression_predictions.items():
    residual = y_validation.to_numpy() - prediction; slug = slugs[name]; errors[name] = abs(residual)
    regression_finalists[[f"MOS_pred_{slug}", f"residual_{slug}", f"absolute_error_{slug}"]] = np.c_[
        prediction, residual, abs(residual)]
regression_error_long = pd.DataFrame(errors).melt(var_name="model", value_name="absolute_error")
paired_errors = pd.DataFrame({"RF − Ridge": errors["Random Forest"] - errors["Ridge"],
    "RF − MLP": errors["Random Forest"] - errors["MLP PyTorch"]}).melt(var_name="comparison",
    value_name="absolute_error_difference")
band_metrics = regression_validation.groupby("MOS_band", observed=True).agg(n=("MOS", "size"),
    MAE=("absolute_error", "mean"), bias_observed_minus_predicted=("residual", "mean"))
scene_long = regression_validation.melt(id_vars=["MOS", "absolute_error", "residual"], value_vars=SCENES,
    var_name="scene", value_name="weight").query("weight > 0")
scene_metrics = scene_long.groupby("scene", sort=False).agg(n=("MOS", "size"), MAE=("absolute_error", "mean"),
    bias_observed_minus_predicted=("residual", "mean")).reset_index()
scene_metrics["prevalence"] = scene_metrics.n / len(validation)
knn_prediction, knn_scores = classification_predictions["KNN"], classification_scores["KNN"]
classification_validation = class_table(validation, Y_validation, knn_prediction, knn_scores)
precision, recall, f1, support = skm.precision_recall_fscore_support(Y_validation, knn_prediction, average=None,
    zero_division=0)
label_metrics = pd.DataFrame({"scene": SCENES, "support_positive": support, "prevalence": Y_validation.mean(0),
    "precision": precision, "recall": recall, "F1": f1,
    "AUC": [skm.roc_auc_score(Y_validation[:, i], knn_scores[:, i]) for i in range(9)],
    "TP": ((Y_validation == 1) & (knn_prediction == 1)).sum(0),
    "FP": ((Y_validation == 0) & (knn_prediction == 1)).sum(0),
    "FN": ((Y_validation == 1) & (knn_prediction == 0)).sum(0)})
threshold_metrics = pd.concat([pd.DataFrame({"scene": SCENES, "threshold": threshold,
    "precision": p, "recall": r, "F1": f}) for threshold in (.3, .5, .7)
    for p, r, f, _ in [skm.precision_recall_fscore_support(Y_validation, knn_scores >= threshold,
        average=None, zero_division=0)]], ignore_index=True)
with parallel_backend("threading", n_jobs=WORKERS):
    permutation = inspection.permutation_importance(regression_models["Random Forest"], X_validation, y_validation,
        n_repeats=20, random_state=SEED, scoring="neg_root_mean_squared_error", n_jobs=-1)
importance = pd.DataFrame({"predictor": PREDICTORS, "permutation_RMSE_mean": permutation.importances_mean,
    "permutation_RMSE_sd": permutation.importances_std,
    "spearman_rho": [spearmanr(X_validation[x], y_validation).statistic for x in PREDICTORS]})
importance["family"] = importance.predictor.map(FAMILIES)
importance.sort_values("permutation_RMSE_mean", ascending=False, inplace=True)
correlation_matrix = X_validation.corr(method="spearman")
```

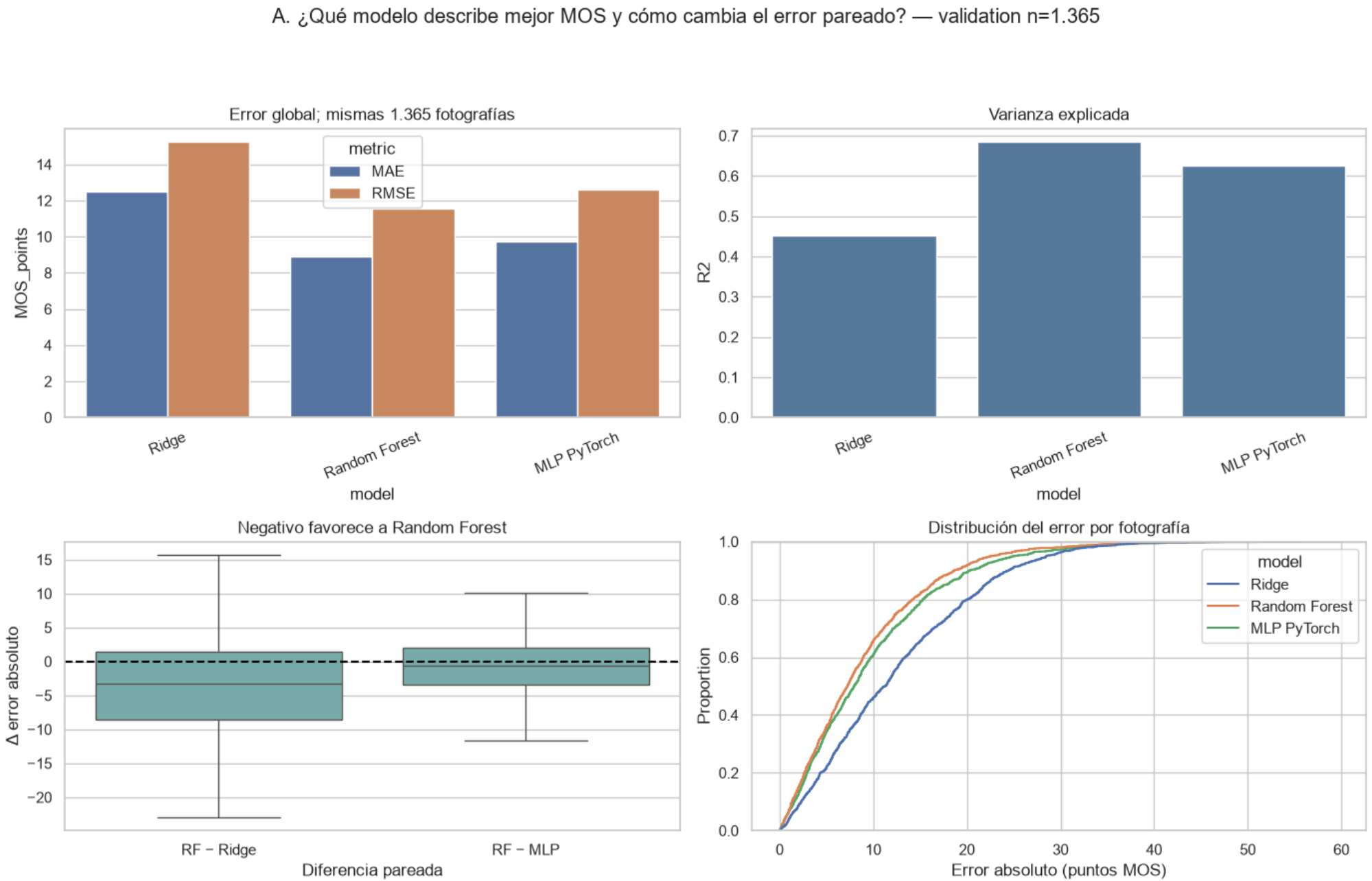
```
ablation_sets = {"All 14 predictors": PREDICTORS, "Without SG2 derived": [x for x in PREDICTORS if x not in
    {"heterogeneity_ratio", "luma_p95_minus_p05"}], **{f"Without {family}": [x for x in PREDICTORS if
    FAMILIES[x] != family] for family in sorted(FAMILY_GROUPS)}}
ablation = pd.DataFrame([{"specification": name, "n_predictors": len(columns), "RMSE_Ridge": skm.mean_squared_error(
    y_validation, pipeline.make_pipeline(impute.SimpleImputer(strategy="median"), preprocessing.RobustScaler(),
        linear_model.Ridge(alpha=ridge.best_params_["ridge__alpha"])).fit(train[columns], y_train).predict(
        validation[columns])) ** .5} for name, columns in ablation_sets.items()])
reference_rmse = ablation.query("specification == 'All 14 predictors'").RMSE_Ridge.iloc[0]
ablation["RMSE_change_vs_full_Ridge"] = ablation.RMSE_Ridge - reference_rmse
clusters_train = train[["image_id", "sha256", "split", "MOS", *SCENES, *TARGETS]].copy().assign(
    cluster_kmeans=kmeans_labels, cluster_dbscan=dbscan_labels, PCA1=pca_coordinates[:, 0], PCA2=pca_coordinates[:, 1])
def posthoc(column):
    grouped = clusters_train.groupby(column); summary = grouped.MOS.agg(n="size", MOS_mean="mean", MOS_median="median")
    return summary.join(grouped[TARGETS].mean().rename(columns=lambda x: x.replace("target_", "prevalence_")))
kmeans_posthoc, dbscan_posthoc = posthoc("cluster_kmeans"), posthoc("cluster_dbscan")
regression_validation["cluster_kmeans_posthoc"] = kmeans.predict(cluster_preprocessor.transform(X_validation))
regression_validation["cluster_assignment_source"] = "KMeans.predict after train fit"
rules = [("validation_lowest_MOS", "MOS", True), ("validation_highest_MOS", "MOS", False),
    ("validation_most_negative_residual", "residual", True),
    ("validation_most_positive_residual", "residual", False),
    ("validation_smallest_absolute_error", "absolute_error", True)]
validation_cases = pd.DataFrame([regression_validation.sort_values([column, "image_id"],
    ascending=[ascending, True]).iloc[0] for _, column, ascending in rules]); validation_cases["selection_rule"] = [x[0] for x in rules]
validation_cases["cluster_dbscan"] = pd.NA
train_cases = clusters_train.assign(MOS_predicted=regression_models["Random Forest"].predict(X_train))
train_cases["residual"] = train_cases.MOS - train_cases.MOS_predicted; train_cases["absolute_error"] = abs(train_cases.residual)
prototypes = []
for label in range(selected_k):
    candidates = train_cases.loc[kmeans_labels == label].copy()
    candidates["centroid_distance"] = np.sum((Z_train[kmeans_labels == label] - kmeans.cluster_centers_[label]) ** 2, 1)
    row = candidates.sort_values(["centroid_distance", "image_id"]).iloc[0].copy()
    row["selection_rule"] = f"train_nearest_KMeans_centroid_{label}"; prototypes.append(row)
prototype_cases = pd.DataFrame(prototypes); prototype_cases["cluster_kmeans_posthoc"] = prototype_cases.cluster_kmeans
prototype_cases["cluster_assignment_source"] = "KMeans fitted train label"
gallery = pd.concat([validation_cases, prototype_cases], ignore_index=True, sort=False)
gallery["active_scenes"] = gallery.apply(lambda row: "|".join(x for x in SCENES if row[x] > 0), axis=1)
gallery["relative_jpeg_path"] = gallery.image_id.map(lambda x: f"spaq/SPAQ/TestImage/{x}.jpg")
```

Visualización para el análisis

In [8]

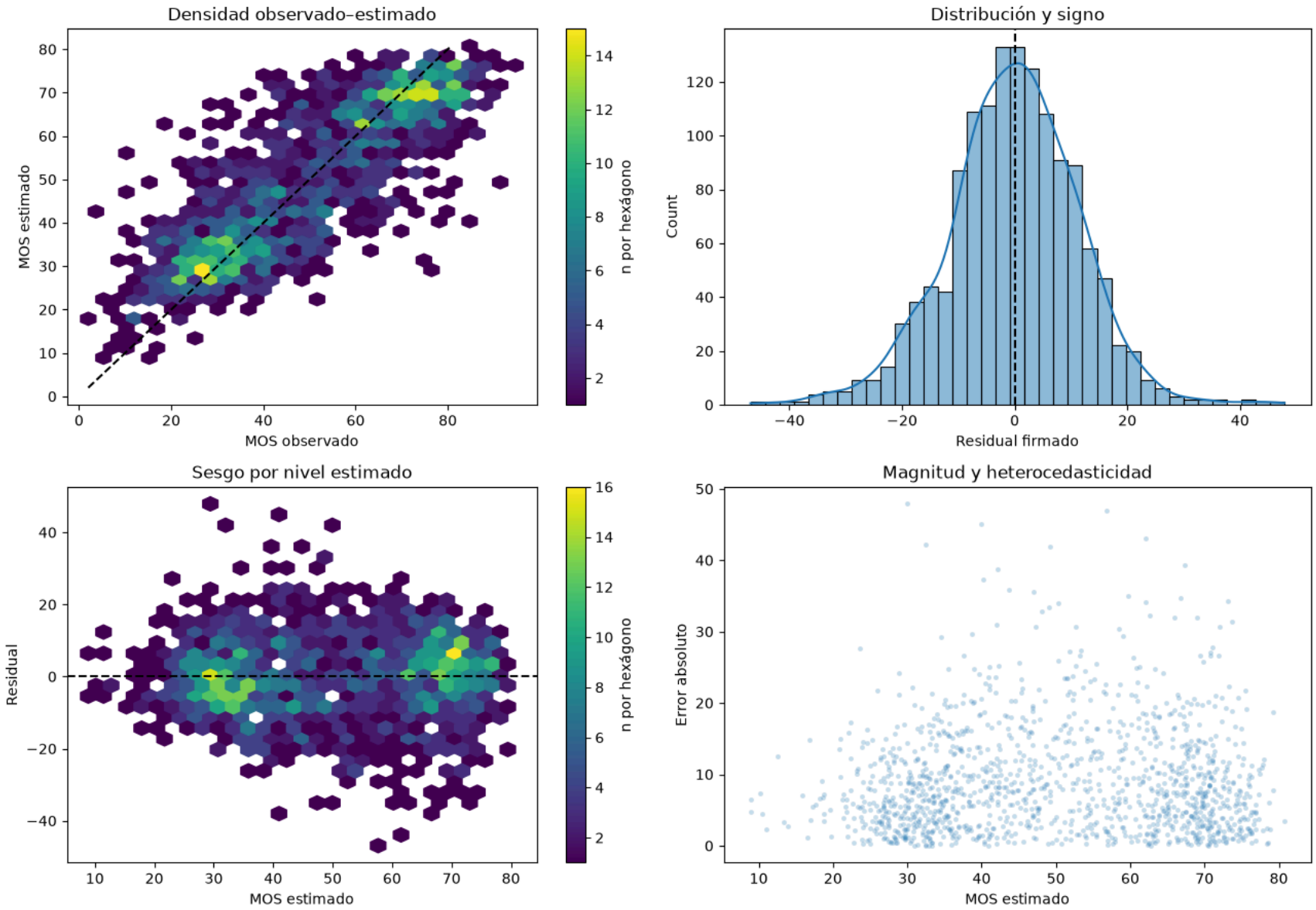
```
def bar(axis, data, x, y, title, rotation=0, **kwargs):
    sns.barplot(data=data, x=x, y=y, ax=axis, **kwargs).set_title(title); axis.tick_params(axis="x", rotation=rotation)
def scatter_pca(axis, data, hue, palette, title):
    sns.scatterplot(data=data, x="PCA1", y="PCA2", hue=hue, palette=palette, s=8, alpha=.35, linewidth=0, ax=axis)
    axis.set(xlabel=f"PC1 ({pca_two.explained_variance_ratio_[0]:.1%})",
        ylabel=f"PC2 ({pca_two.explained_variance_ratio_[1]:.1%})", title=title); axis.legend(fontsize=8)
figure, axes = plt.subplots(2, 2, figsize=(14, 9))
metrics_long = regression_summary.melt("model", value_vars=["MAE", "RMSE"], var_name="metric", value_name="MOS_points")
bar(axes[0, 0], metrics_long, "model", "MOS_points", "Error global; mismas 1.365 fotografías", 22, hue="metric")
bar(axes[0, 1], regression_summary, "model", "R2", "Varianza explicada", 22, color="#4C78A8")
sns.boxplot(data=paired_errors, x="comparison", y="absolute_error_difference", showfliers=False,
    color="#72B7B2", ax=axes[1, 0]); axes[1, 0].axhline(0, color="black", ls="--")
axes[1, 0].set(xlabel="Diferencia pareada", ylabel="Δ error absoluto", title="Negativo favorece a Random Forest")
sns.ecdfplot(data=regression_error_long, x="absolute_error", hue="model", ax=axes[1, 1])
axes[1, 1].set(xlabel="Error absoluto (puntos MOS)", title="Distribución del error por fotografía")
```

```
save(figure, "A_regresion_global_validation",
    "A. ¿Qué modelo describe mejor MOS y cómo cambia el error pareado? — validation n=1.365")
```

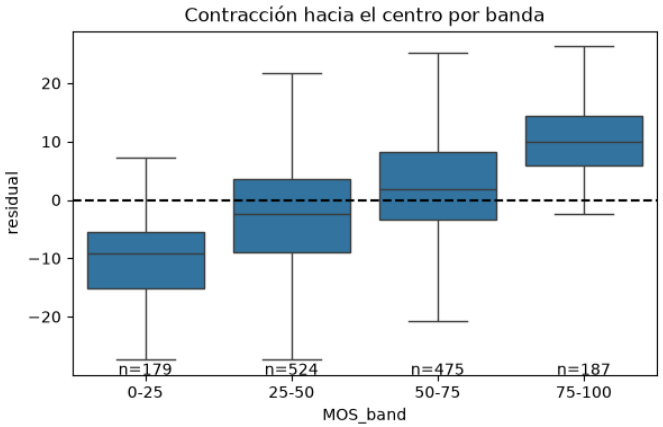


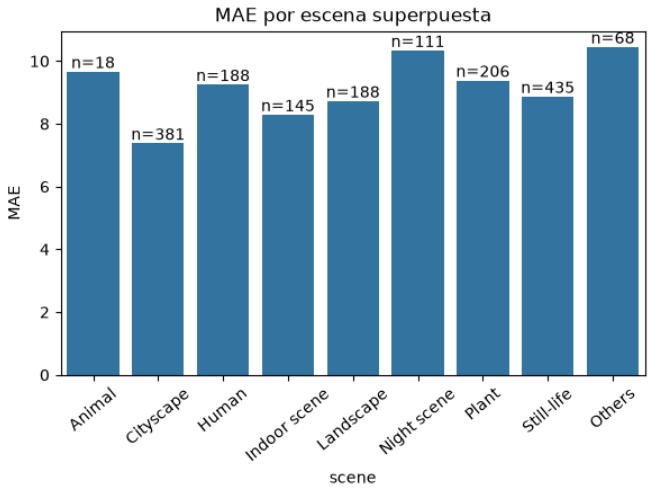
In [9]

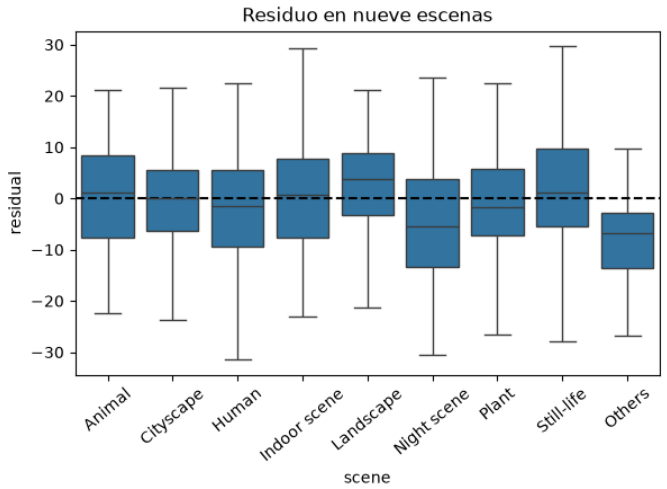
```
figure, axes = plt.subplots(2, 2, figsize=(13, 10))
hexbin = axes[0, 0].hexbin(regression_validation.MOS, regression_validation.MOS_predicted, gridsize=28, mincnt=1)
limits = regression_validation[["MOS", "MOS_predicted"]].agg(["min", "max"]).to_numpy().ravel()[[0, 3]]
axes[0, 0].plot(limits, limits, "k--"); axes[0, 0].set(xlabel="MOS observado", ylabel="MOS estimado",
    title="Densidad observado–estimado"); figure.colorbar(hexbin, ax=axes[0, 0], label="n por hexágono")
sns.histplot(regression_validation, x="residual", kde=True, ax=axes[0, 1]); axes[0, 1].axvline(0, color="black", ls="--")
axes[0, 1].set(xlabel="Residual firmado", title="Distribución y signo")
hexbin = axes[1, 0].hexbin(regression_validation.MOS_predicted, regression_validation.residual, gridsize=28, mincnt=1)
axes[1, 0].axhline(0, color="black", ls="--"); axes[1, 0].set(xlabel="MOS estimado", ylabel="Residual",
    title="Sesgo por nivel estimado"); figure.colorbar(hexbin, ax=axes[1, 0], label="n por hexágono")
sns.scatterplot(data=regression_validation, x="MOS_predicted", y="absolute_error", s=12, alpha=.25, ax=axes[1, 1])
axes[1, 1].set(xlabel="MOS estimado", ylabel="Error absoluto", title="Magnitud y heterocedasticidad")
save(figure, "B1_diagnostico_regresion_validation", "B1. ¿Dónde falla Random Forest? — validation n=1.365")
figure, axes = plt.subplots(1, 3, figsize=(18, 5.2))
sns.boxplot(data=regression_validation, x="MOS_band", y="residual", showfliers=False, ax=axes[0]); axes[0].axhline(0, c="k", ls="--")
axes[0].set(title="Contracción hacia el centro por banda"); bar(axes[1], scene_metrics, "scene", "MAE",
    "MAE por escena superpuesta", 40); sns.boxplot(data=scene_long, x="scene", y="residual", showfliers=False, ax=axes[2])
for index, row in band_metrics.reset_index().iterrows(): axes[0].text(index, axes[0].get_ylim()[0], f"n={int(row.n)}", ha="center")
for index, row in scene_metrics.iterrows(): axes[1].text(index, row.MAE, f"n={int(row.n)}", ha="center", va="bottom")
axes[2].axhline(0, c="k", ls="--"); axes[2].tick_params(axis="x", rotation=40); axes[2].set(title="Residuo en nueve escenas")
save(figure, "B2_error_condicionado_validation", "B2. Error por MOS y contenido — validation; escenas no exclusivas")
```


B1. ¿Dónde falla Random Forest? — validation n=1.365
Densidad observado–estimado
MOS estimado
MOS observado
n por hexágono
Distribución y signo
Count
Residual firmado
Sesgo por nivel estimado
Residual
MOS estimado
n por hexágono
Magnitud y heterocedasticidad
Error absoluto
MOS estimado

B2. Error por MOS y contenido — validation; escenas no exclusivas



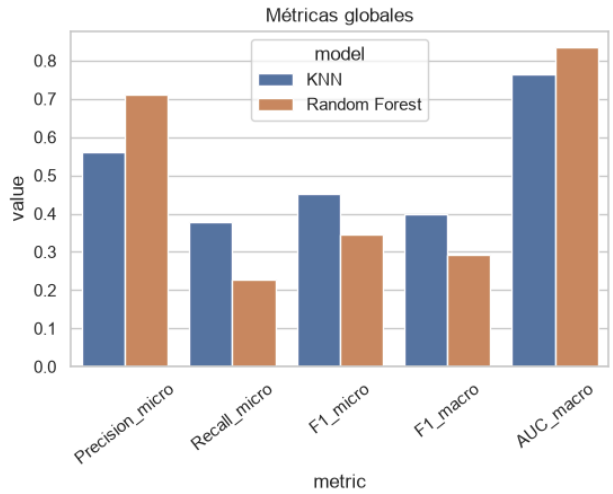


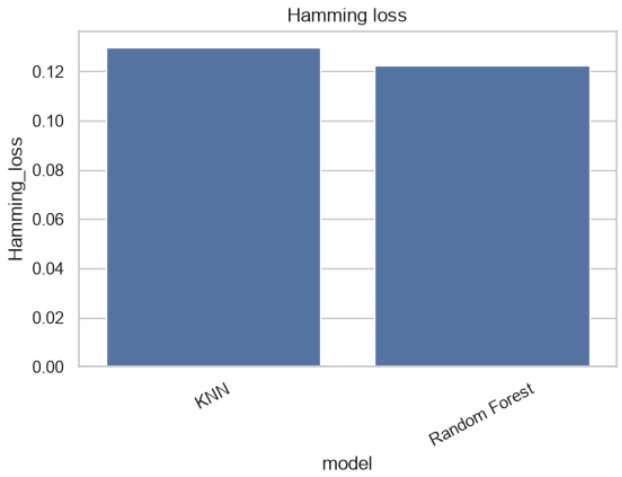


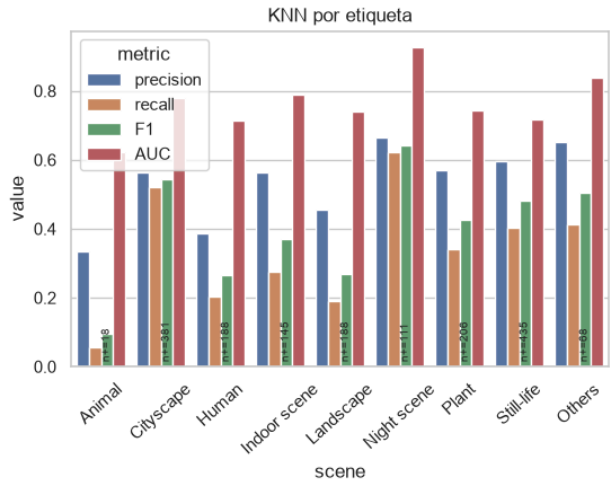
In [10]

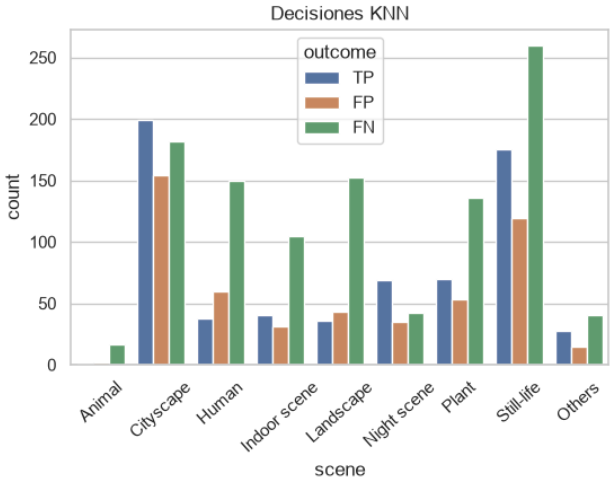
```
figure, axes = plt.subplots(2, 3, figsize=(18, 10))
global_long = classification_summary.melt("model", value_vars=["Precision_micro", "Recall_micro", "F1_micro",
    "F1_macro", "AUC_macro"], var_name="metric", value_name="value")
bar(axes[0, 0], global_long, "metric", "value", "Métricas globales", 38, hue="model")
bar(axes[0, 1], classification_summary, "model", "Hamming_loss", "Hamming loss", 28)
label_long = label_metrics.melt(["scene", "support_positive", "prevalence"], value_vars=["precision", "recall", "F1", "AUC"],
    var_name="metric", value_name="value"); bar(axes[0, 2], label_long, "scene", "value", "KNN por etiqueta", 42, hue="metric")
for index, row in label_metrics.iterrows(): axes[0, 2].text(index, .02, f"n+={row.support_positive}", rotation=90, fontsize=7)
confusion = label_metrics.melt("scene", value_vars=["TP", "FP", "FN"], var_name="outcome", value_name="count")
bar(axes[1, 0], confusion, "scene", "count", "Decisiones KNN", 42, hue="outcome")
bar(axes[1, 1], label_metrics, "scene", "prevalence", "Prevalencia", 42)
for index, row in label_metrics.iterrows(): axes[1, 1].text(index, row.prevalence, f"n+={row.support_positive}", ha="center", va="bottom", fontsize=7)
sns.heatmap(threshold_metrics.pivot(index="scene", columns="threshold", values="F1"), vmin=0, vmax=1,
    annot=True, fmt=".2f", ax=axes[1, 2]); axes[1, 2].set(title="Sensibilidad de F1 al umbral")
save(figure, "C_clasificacion_multietiqueta_validation",
    "C. Frecuencia, umbral y error en nueve etiquetas — validation n=1.365")
figure, axes = plt.subplots(2, 2, figsize=(15, 10)); pca_plot = pd.DataFrame(pca_coordinates, columns=["PCA1", "PCA2"])
for axis, column, ylabel in ((axes[0, 0], "inertia", "Inercia"), (axes[0, 1], "silhouette", "Silueta")):
    axis.plot(k_selection.K, k_selection[column], marker="o"); axis.set(xlabel="K", ylabel=ylabel, title=f"{ylabel}: K=2–8")
axes[0, 1].axvline(selected_k, c="k", ls="--")
pca_plot["K-Means"] = pd.Series(kmeans_labels).map({x: f"Grupo {x}; n={(kmeans_labels == x).sum()}" for x in range(3)})
scatter_pca(axes[1, 0], pca_plot, "K-Means", "colorblind", "K-Means en PCA de train")
loadings = pca_loadings[["PC1", "PC2"]].reset_index(names="predictor").melt("predictor", var_name="component", value_name="loading")
sns.barplot(data=loadings, x="loading", y="predictor", hue="component", ax=axes[1, 1]); axes[1, 1].set(title="Cargas PCA")
save(figure, "D1_kmeans_pca_train", "D1. Organización de 14 predictores — train n=8.364")
```

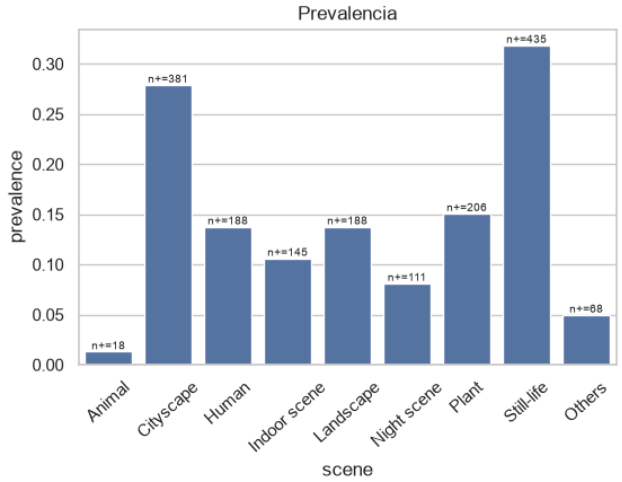
C. Frecuencia, umbral y error en nueve etiquetas — validation n=1.365

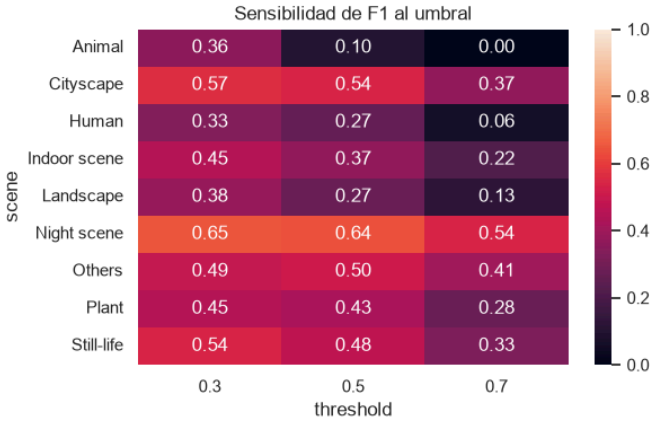




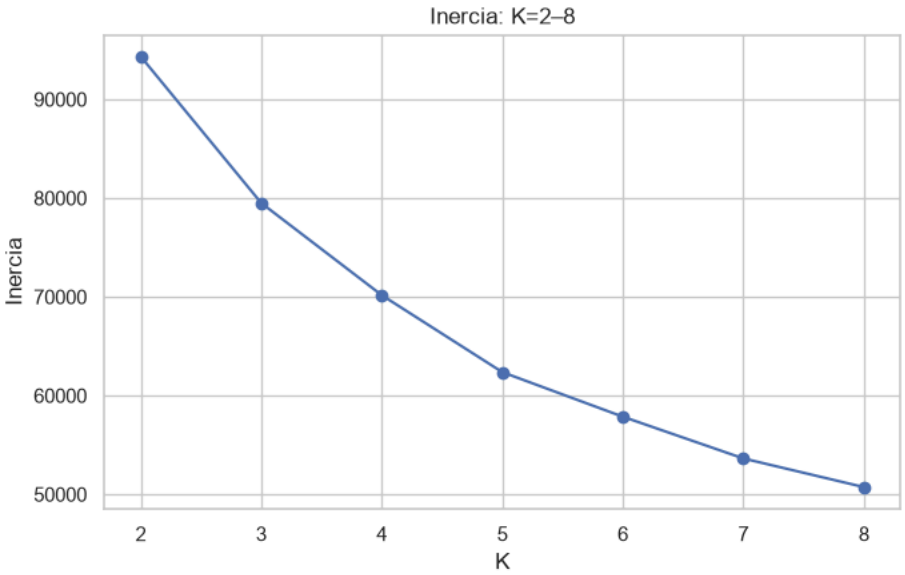


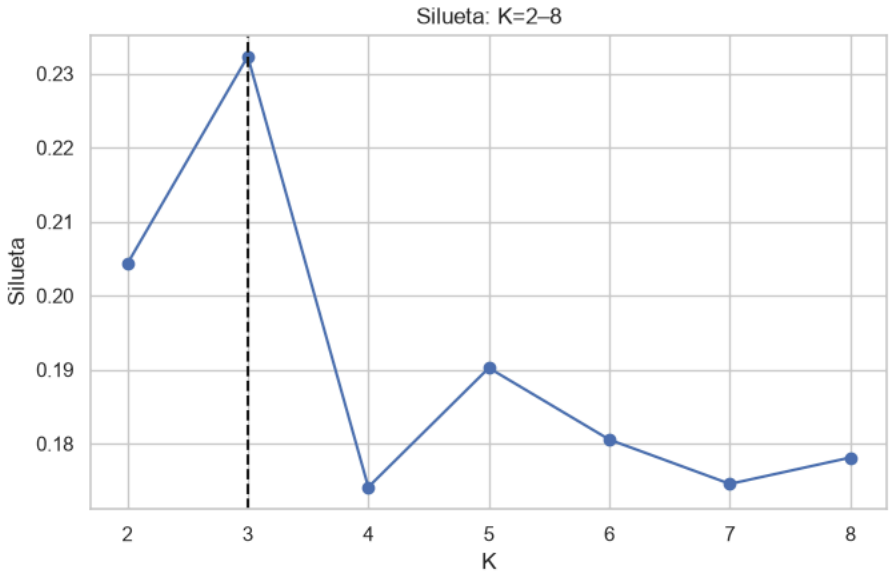


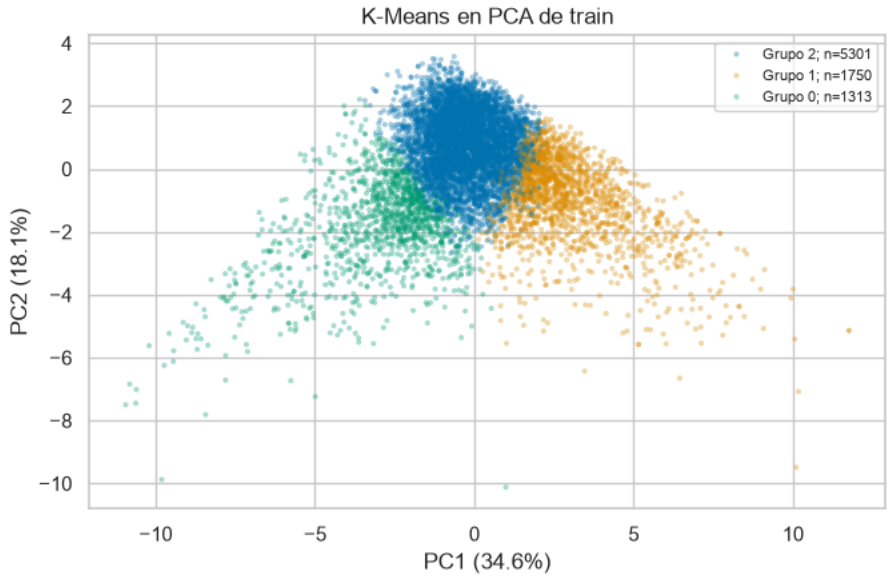


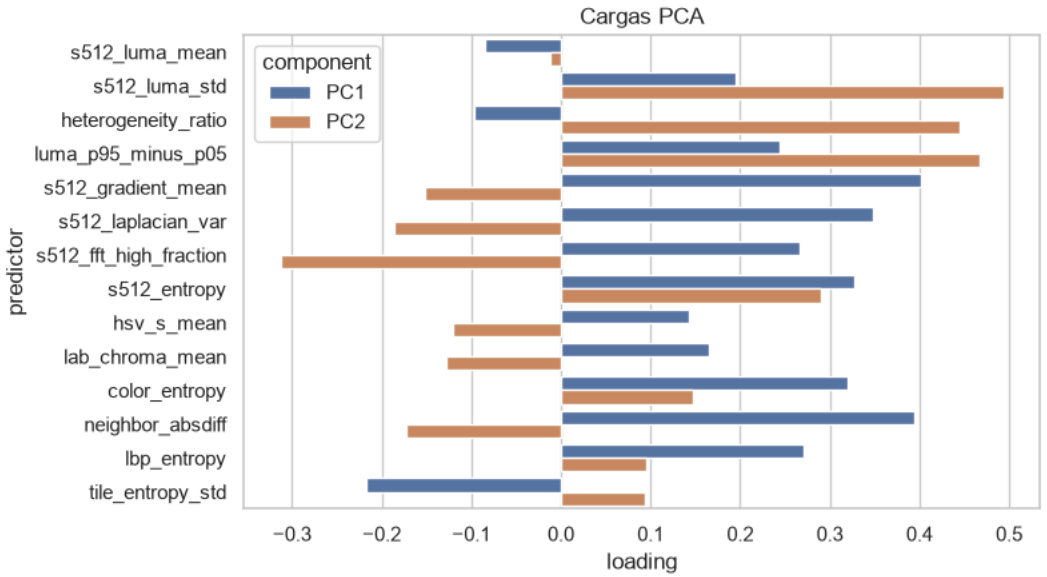


D1. Organización de 14 predictores — train n=8.364





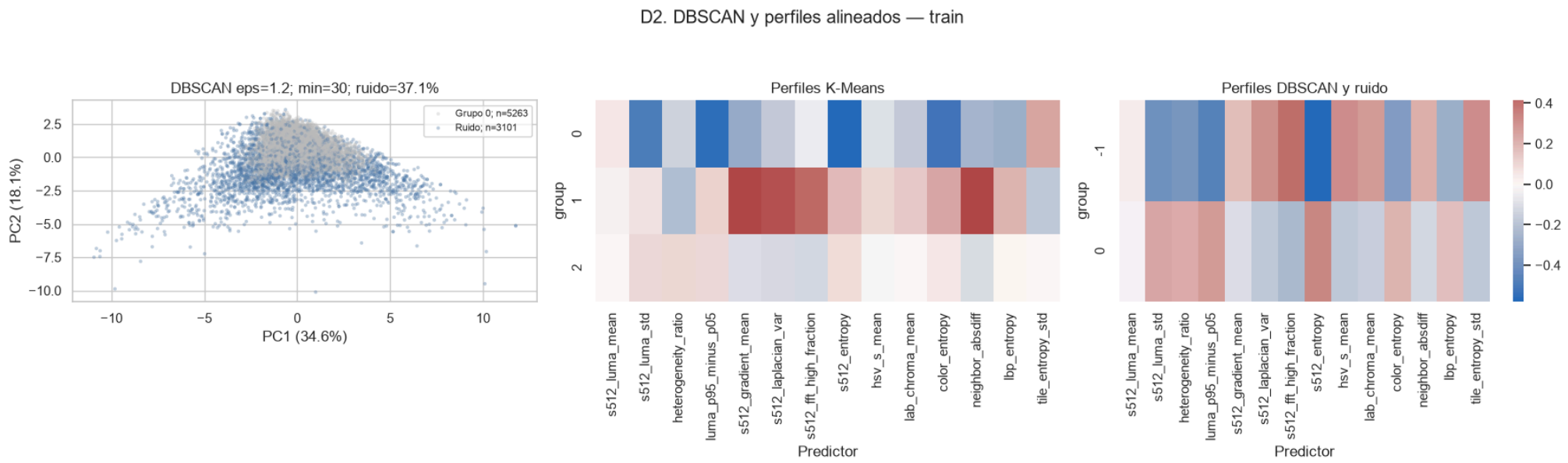


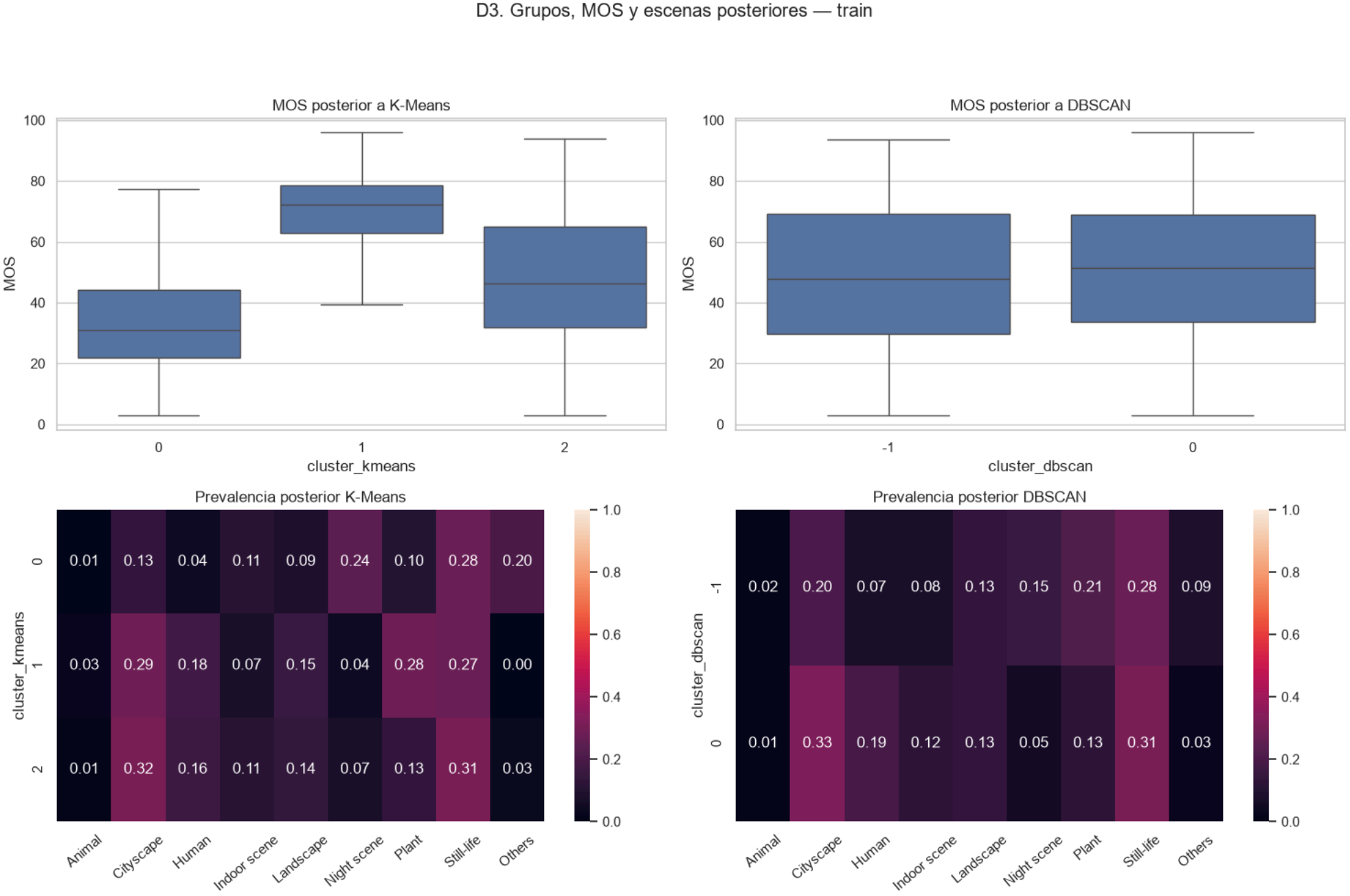


In [11]

```
figure, axes = plt.subplots(1, 3, figsize=(18, 5.3)); values = sorted(set(dbscan_labels))
pca_plot["DBSCAN"] = pd.Series(dbscan_labels).map({x: f"{'Ruido' if x == -1 else f'Grupo {x}'}; n={(dbscan_labels == x).sum()}" for x in values})
```

```
scatter_pca(axes[0], pca_plot, "DBSCAN", ["#BDBDBD", "#4C78A8"],
    f"DBSCAN eps={choice.eps:g}; min={int(choice.min_samples)}; ruido={choice.noise_fraction:.1%}")
z_columns = [f"z_{x}" for x in PREDICTORS]
for axis, frame, title in ((axes[1], kmeans_profiles, "Perfiles K-Means"), (axes[2], dbscan_profiles, "Perfiles DBSCAN y ruido")):
    sns.heatmap(frame[z_columns].set_axis(PREDICTORS, axis=1), center=0, cmap="vlag", ax=axis, cbar=axis is axes[2])
    axis.set(xlabel="Predictor", title=title)
save(figure, "D2_dbscan_perfiles_train", "D2. DBSCAN y perfiles alineados — train")
figure, axes = plt.subplots(2, 2, figsize=(15, 10)); prevalence = [f"prevalence_{x}" for x in SCENES]
for axis, column, title in ((axes[0, 0], "cluster_kmeans", "MOS posterior a K-Means"),
    (axes[0, 1], "cluster_dbscan", "MOS posterior a DBSCAN")):
    sns.boxplot(data=clusters_train, x=column, y="MOS", showfliers=False, ax=axis); axis.set_title(title)
for axis, frame, title in ((axes[1, 0], kmeans_posthoc, "Prevalencia posterior K-Means"),
    (axes[1, 1], dbscan_posthoc, "Prevalencia posterior DBSCAN")):
    sns.heatmap(frame[prevalence].set_axis(SCENES, axis=1), vmin=0, vmax=1, annot=True, fmt=".2f", ax=axis)
    axis.set_title(title); axis.tick_params(axis="x", rotation=40)
save(figure, "D3_posthoc_clusters_train", "D3. Grupos, MOS y escenas posteriores — train")
```



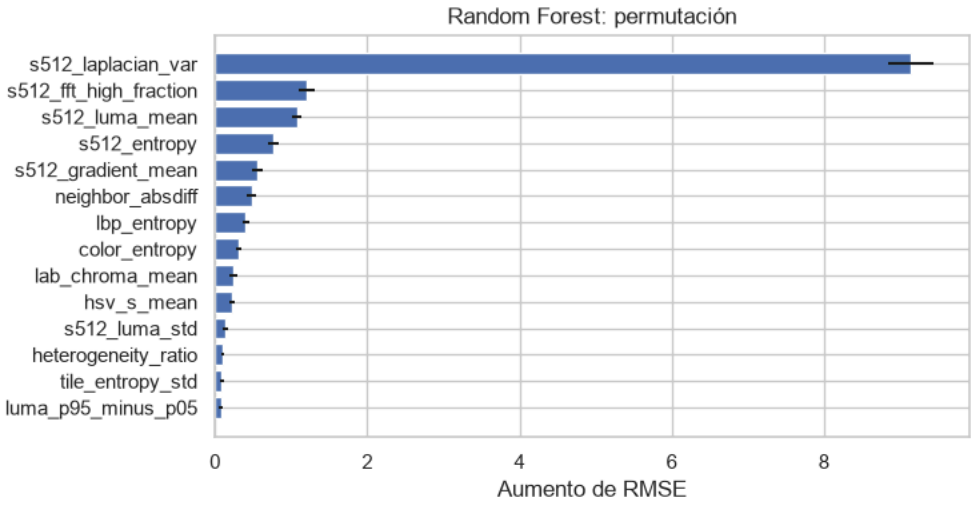


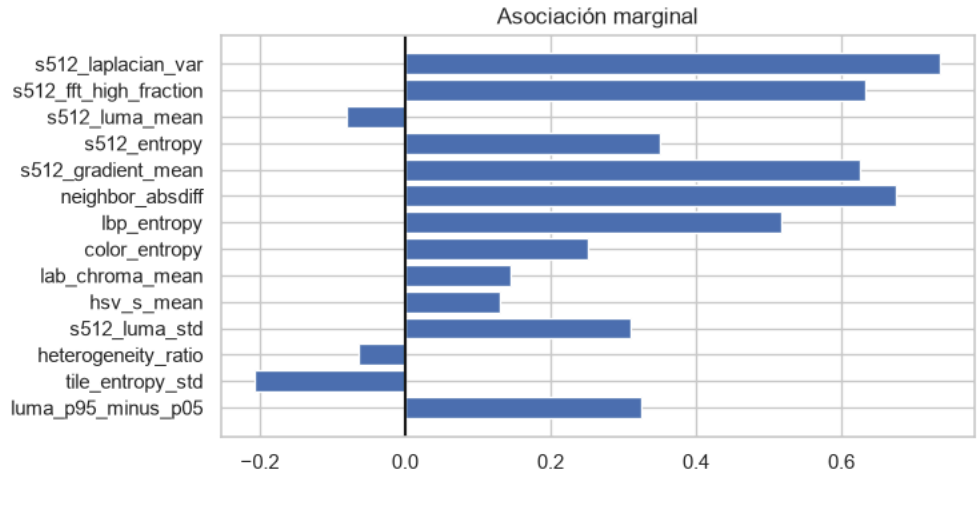
In [12]:

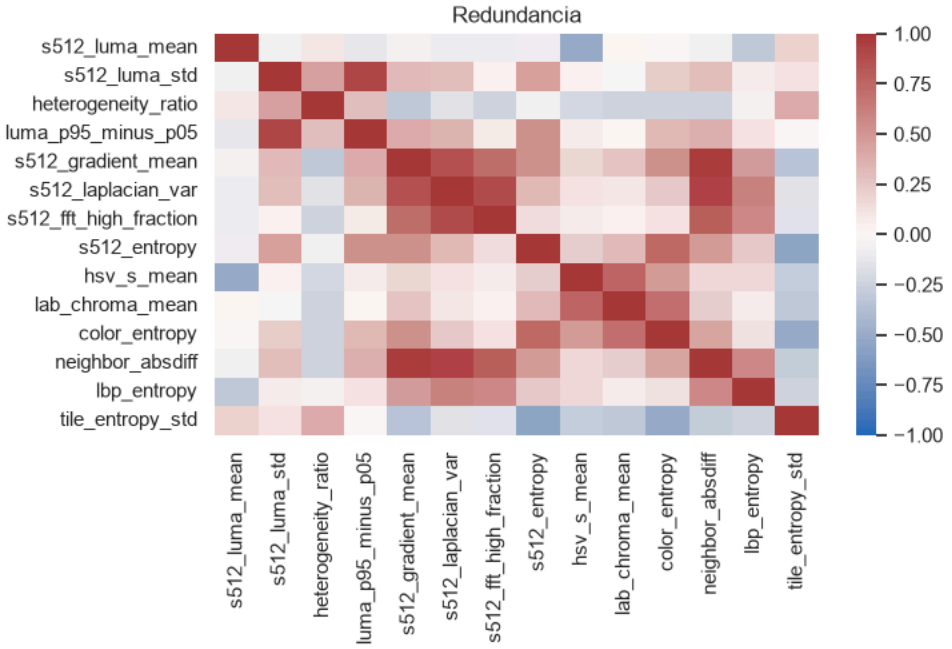
```
figure, axes = plt.subplots(2, 2, figsize=(15, 10)); axes = axes.ravel(); ordered = importance.sort_values("permutation_RMSE_mean")
axes[0].barh(ordered.predictor, ordered.permutation_RMSE_mean, xerr=ordered.permutation_RMSE_sd)
```

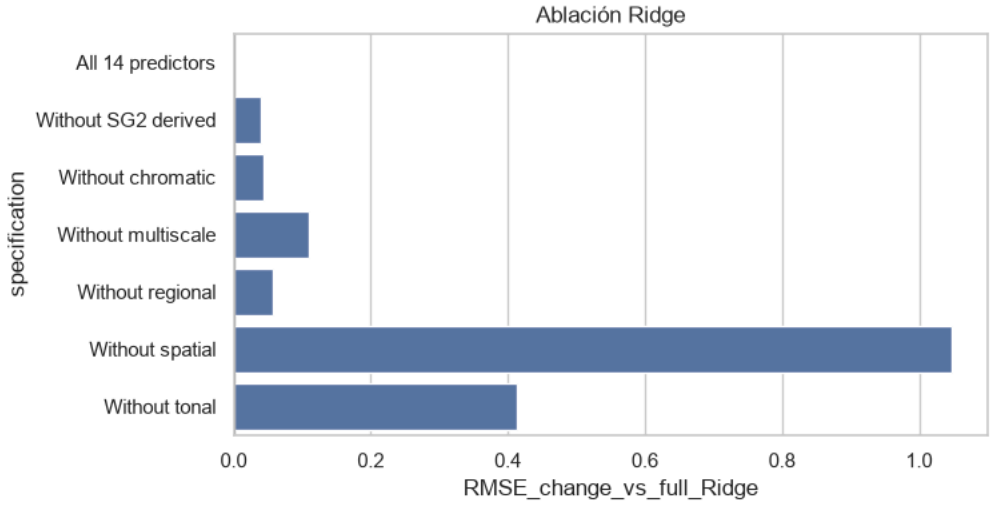
```
axes[0].set(title="Random Forest: permutación", xlabel="Aumento de RMSE")
axes[1].barh(ordered.predictor, ordered.spearman_rho); axes[1].axvline(0, c="k"); axes[1].set(title="Asociación marginal")
sns.heatmap(correlation_matrix, vmin=-1, vmax=1, center=0, cmap="vlag", ax=axes[2]); axes[2].set_title("Redundancia")
bar(axes[3], ablation, "RMSE_change_vs_full_Ridge", "specification", "Ablación Ridge"); axes[3].axvline(0, c="k")
save(figure, "E_interpretabilidad_validation", "E. Asociación, importancia RF y ablación Ridge — validation")
figure, axes = plt.subplots(1, 2, figsize=(12.5, 4.5)); zoom = history.query("epoch >= @best_epoch - 80")
for axis, frame, title in ((axes[0], history, "Trayectoria completa"), (axes[1], zoom, f"Mejor época = {best_epoch}")):
    axis.plot(frame.epoch, frame.train_mse, label="Train interno"); axis.plot(frame.epoch, frame.stop_mse, label="Validación interna")
    axis.axvline(best_epoch, c="k", ls="--"); axis.set(xlabel="Época", ylabel="MSE", title=title); axis.legend()
save(figure, "F_mlp_perdida_train", "F. Convergencia del MLP")
```

E. Asociación, importancia RF y ablación Ridge — validation

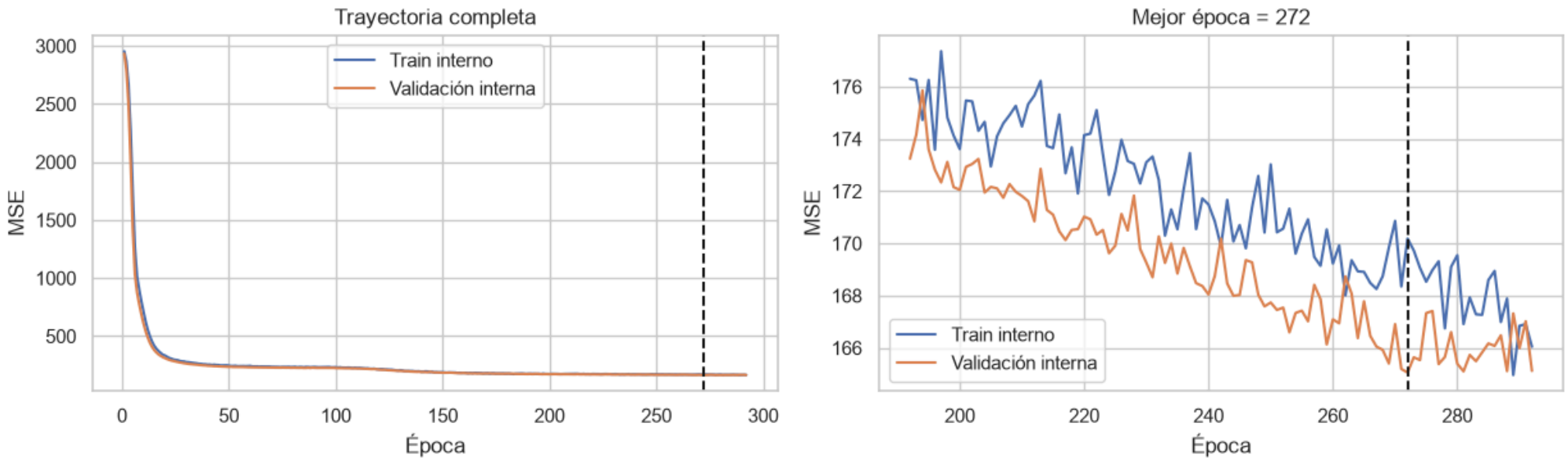








F. Convergencia del MLP



In [13]

```
def gallery_plot(cases, name, title):
    figure, axes = plt.subplots(1, len(cases), figsize=(4 * len(cases), 5.5))
    for axis, (_, row) in zip(np.atleast_1d(axes), cases.iterrows()):
        with Image.open(IMAGES / f"{row.image_id}.jpg") as raw: axis.imshow(ImageOps.exif_transpose(raw).convert("RGB"))
        db = "NA" if pd.isna(row.cluster_dbscan) else int(row.cluster_dbscan)
        axis.set_title(f"{row.selection_rule}\nID {row.image_id} | {row.split}\nMOS={row.MOS:.1f} | pred={row.MOS_predicted:.1f} | "
            f"res={row.residual:+.1f}\nescenas={row.active_scenes}\nK-Means={int(row.cluster_kmeans_posthoc)} | DBSCAN={db}", fontsize=8)
        axis.axis("off")
    save(figure, name, title, .86)
gallery_plot(gallery.query("split == 'validation'"), "G1_casos_reales_validation",
    "G1. Casos reales por reglas predefinidas — validation; K-Means post hoc")
gallery_plot(gallery.query("split == 'train'"), "G2_prototipos_reales_train",
    "G2. Fotografías reales próximas a centroides K-Means — train")
```

G1. Casos reales por reglas predefinidas — validation; K-Means post hoc

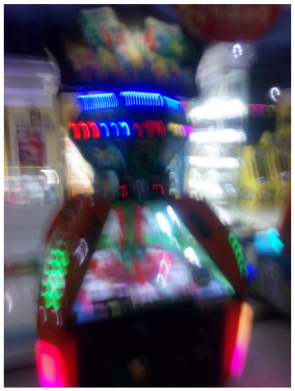
validation_lowest_MOS
ID 07338 | validation
MOS=2.0 | pred=16.9 | res=-14.9
escenas=Others
K-Means=0 | DBSCAN=NA



validation_highest_MOS
ID 07296 | validation
MOS=94.7 | pred=75.6 | res=+19.1
escenas=Cityscape|Still-life
K-Means=2 | DBSCAN=NA



validation_most_negative_residual
ID 04623 | validation
MOS=9.9 | pred=56.8 | res=-46.9
escenas=Night scene
K-Means=2 | DBSCAN=NA



validation_most_positive_residual
ID 07372 | validation
MOS=77.8 | pred=29.9 | res=+47.9
escenas=Landscape
K-Means=2 | DBSCAN=NA



validation_smallest_absolute_error
ID 09913 | validation
MOS=32.5 | pred=32.5 | res=+0.0
escenas=Still-life
K-Means=2 | DBSCAN=NA



G2. Fotografías reales próximas a centroides K-Means — train

train_nearest_KMeans_centroid_0
ID 05176 | train
MOS=34.6 | pred=32.9 | res=+1.7
escenas=Indoor scene
K-Means=0 | DBSCAN=0



train_nearest_KMeans_centroid_1
ID 01317 | train
MOS=75.3 | pred=74.6 | res=+0.7
escenas=Still-life
K-Means=1 | DBSCAN=0



train_nearest_KMeans_centroid_2
ID 09715 | train
MOS=42.5 | pred=40.9 | res=+1.6
escenas=Cityscape|Human|Plant
K-Means=2 | DBSCAN=0



Test histórico, exportación y controles

In [14]

```
test_ids = set(test_sealed.image_id)
test = annotations(FILES["scores"], test_ids).merge(annotations(FILES["scenes"], test_ids),
    on=["image_id", "Image name"], validate="one_to_one").merge(test_sealed, on="image_id")
test[TARGETS] = (test[SCENES] > 0).astype("int8").set_axis(TARGETS, axis=1)
X_test, y_test, Y_test = test[PREDICTORS], test.MOS, test[TARGETS].to_numpy()
development_full = pd.concat([train, validation]); X_development = development_full[PREDICTORS]
final_regressor = base.clone(regression_models["Random Forest"]).fit(X_development, development_full.MOS)
final_classifier = base.clone(classification_models["KNN"]).fit(X_development, development_full[TARGETS].to_numpy())
```

```
test_regression_prediction = final_regressor.predict(X_test); test_classification_prediction = final_classifier.predict(X_test)
test_classification_scores = scores(final_classifier, X_test)
regression_test = reg_table(test, test_regression_prediction); classification_test = class_table(test, Y_test,
    test_classification_prediction, test_classification_scores)
for frame in (regression_test, classification_test): frame["status"] = "historical_reproducibility_frozen_decisions"
summary_models = pd.concat([regression_summary.assign(task="regression", split="validation"),
    classification_summary.assign(task="multilabel_classification", split="validation")], ignore_index=True)
gallery_columns = ["image_id", "sha256", "split", "selection_rule", "relative_jpeg_path", "MOS", "MOS_predicted",
    "residual", "absolute_error", "active_scenes", "cluster_kmeans_posthoc", "cluster_dbscan", "cluster_assignment_source"]
tables = {"predicciones_regresion_validacion.csv": regression_validation,
    "predicciones_finalistas_validacion.csv": regression_finalists,
    "diferencias_error_pareadas_validacion.csv": paired_errors,
    "predicciones_regresion_test_reproducibilidad.csv": regression_test,
    "clasificacion_validacion.csv": classification_validation, "clasificacion_test_reproducibilidad.csv": classification_test,
    "clusters_entrenamiento.csv": clusters_train, "resumen_modelos.csv": summary_models,
    "importancia_permutacion.csv": importance, "metricas_por_escena.csv": scene_metrics,
    "metricas_clasificacion_por_escena.csv": label_metrics, "ablacion_familias_ridge.csv": ablation,
    "sensibilidad_umbral_clasificacion.csv": threshold_metrics, "seleccion_kmeans.csv": k_selection.drop(columns="model"),
    "seleccion_dbscan.csv": dbscan_selection, "perfil_posthoc_kmeans.csv": kmeans_posthoc.reset_index(),
    "perfil_posthoc_dbscan.csv": dbscan_posthoc.reset_index(), "varianza_pca.csv": pd.DataFrame({"component": ["PC1", "PC2"],
        "explained_variance_ratio": pca_two.explained_variance_ratio_}), "indice_galeria_casos.csv": gallery[gallery_columns],
    "historial_mlp.csv": history}
for filename, frame in tables.items(): frame.to_csv(RESULTS / filename, index=False)
kmeans_profiles.to_csv(RESULTS / "perfiles_clusters.csv", index_label="cluster_kmeans")
dbscan_profiles.to_csv(RESULTS / "perfiles_dbscan.csv", index_label="cluster_dbscan")
band_metrics.to_csv(RESULTS / "metricas_por_banda_mos.csv"); pca_loadings.to_csv(RESULTS / "cargas_pca.csv")
manifest = pd.DataFrame({"order": range(68), "descriptor": DESCRIPTORS}); manifest["is_predictor"] = manifest.descriptor.isin(PREDICTORS)
manifest["family"] = manifest.descriptor.map(FAMILIES); manifest.to_csv(RESULTS / "manifiesto_descriptores.csv", index=False)
metadata[["image_id", "sha256", "split", *DESCRIPTORS, "heterogeneity_ratio", "luma_p95_minus_p05"]].to_csv(
    RESULTS / "descriptores_por_imagen.csv", index=False)
models = {"modelo_regresion_final.joblib": final_regressor, "modelo_clasificacion_final.joblib": final_classifier,
    "transformador_clustering.joblib": cluster_preprocessor, "kmeans.joblib": kmeans, "dbscan.joblib": dbscan,
    "pca_visualizacion.joblib": pca_two, "transformador_mlp.joblib": mlp_preprocessor,
    "dbscan_parametros.joblib": {"eps": float(choice.eps), "min_samples": int(choice.min_samples)}}
for filename, model in models.items(): joblib.dump(model, MODELS / filename)
torch.save({"state_dict": best_state, "predictors": PREDICTORS, "best_epoch": best_epoch}, MODELS / "mlp_train.pt")
test_regression_metrics = reg_metrics(y_test, test_regression_prediction)
test_classification_metrics = class_metrics(Y_test, test_classification_prediction, test_classification_scores)
event = {"timestamp": datetime.now(ZoneInfo("America/Bogota")).isoformat(),
    "status": "historical_reproducibility_frozen_decisions", "frozen_decisions": frozen,
    "regression_test": test_regression_metrics, "classification_test": test_classification_metrics}
(RESULTS / "evento_test_reproducibilidad.json").write_text(json.dumps(event, indent=2), encoding="utf-8")
```

```
1031
```

In [15]:

```
expected_figures = """A_regresion_global_validation B1_diagnostico_regresion_validation B2_error_condicionado_validation
C_clasificacion_multietiqueta_validation D1_kmeans_pca_train D2_dbscan_perfiles_train D3_posthoc_clusters_train
E_interpretabilidad_validation F_mlp_perdida_train G1_casos_reales_validation G2_prototipos_reales_train""".split()
assert np.allclose(regression_summary.set_index("model").loc["Random Forest", ["MAE", "RMSE", "R2"]],
    [8.926698087423688, 11.57665869103917, .6839482656563363])
assert np.isclose(classification_summary.set_index("model").loc["KNN", "F1_micro"], .4511691884456671)
assert kmeans_profiles.n.astype(int).tolist() == [1313, 1750, 5301]
assert dict(zip(*final_counts)) == {-1: 3101, 0: 5263} and np.array_equal(dbscan.labels_, dbscan_labels)
assert all(count >= dbscan.min_samples for label, count in zip(*final_counts) if label != -1)
assert {x.stem for x in FIGURES.glob("*.png")} == set(expected_figures)
assert {x.stem for x in FIGURES.glob("*.svg")} == set(expected_figures)
```

```
closure = {"status": "PASS", "code_cells": 15, "rows": len(metadata), "unique_sha": metadata.sha256.nunique(),
    "split_counts": split_counts.to_dict(), "extraction_minutes": extraction_minutes, "figures_png": 11, "figures_svg": 11,
    "dbscan_parameters": dbscan.get_params(), "dbscan_counts": {str(k): int(v) for k, v in zip(*final_counts)},
    "notebook_inputs": [str(x) for x in FILES.values()] + [str(IMAGES / "*.jpg")],
    "test_status": "historical_reproducibility_frozen_decisions"}
(RESULTS / "control_cierre.json").write_text(json.dumps(closure, indent=2), encoding="utf-8")
display(pd.Series(test_regression_metrics), pd.Series(test_classification_metrics)); print(json.dumps(closure, indent=2))
```

```
MAE      9.050575
RMSE    11.568386
R2       0.696021
dtype: float64

F1_micro           0.425561
F1_macro           0.369052
Precision_micro    0.536118
Recall_micro       0.352806
Hamming_loss       0.132362
Subset_accuracy    0.281519
AUC_macro          0.755733
dtype: float64

{
  "status": "PASS",
  "code_cells": 15,
  "rows": 11125,
  "unique_sha": 10803,
  "split_counts": {
    "train": 8364,
    "test": 1396,
    "validation": 1365
  },
  "extraction_minutes": 11.011356663703918,
  "figures_png": 11,
  "figures_svg": 11,
  "dbscan_parameters": {
    "algorithm": "auto",
    "eps": 1.2,
    "leaf_size": 30,
    "metric": "euclidean",
    "metric_params": null,
    "min_samples": 30,
    "n_jobs": -1,
    "p": null
  },
  "dbscan_counts": {
    "-1": 3101,
    "0": 5263
  },
  "notebook_inputs": [
    "D:\\Data analisys\\spaq\\SPAQ\\Annotations\\MOS and Image attribute scores.xlsx",
    "D:\\Data analisys\\spaq\\SPAQ\\Annotations\\Scene category labels.xlsx",
    "D:\\Data analisys\\spaq\\SPAQ\\Annotations\\EXIF_tags.xlsx",
    "D:\\Data analisys\\spaq\\SPAQ\\TestImage\\*.jpg"
  ],
```

```
  "test_status": "historical_reproducibility_frozen_decisions"
}
```